CYPRUS INTERNATIONAL UNIVERSITY

FACULTY of ENGINEERING

DATABASE MANAGEMENT SYSTEMS AND PROGRAMMING II

STRAWBERRY FARMING CORP.

Students and Roles ;

- **Fatmatül Zehra Taş (22208075):** DDL (table creation, constraints), DML (insert, update, delete operations), database connection (Npgsql), SQL queries, and User Interface (UI) development
- **Hediye Sultan Bozkurt (22319266):** Report writing, PL/pgSQL (procedures, functions, triggers), and advanced SQL queries
- **Zeynep Özlüer (22315460):** ER Diagram (ERD) design, database modeling, and contribution to SQL queries.
- **Emre Özel (22315566):** User Interface (UI) design (WinForms), application layout, and database integration

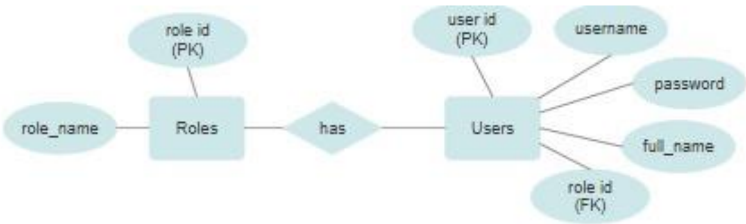
Instructor*: PROF. DR. MELİKE ŞAH DİREKOĞLU*

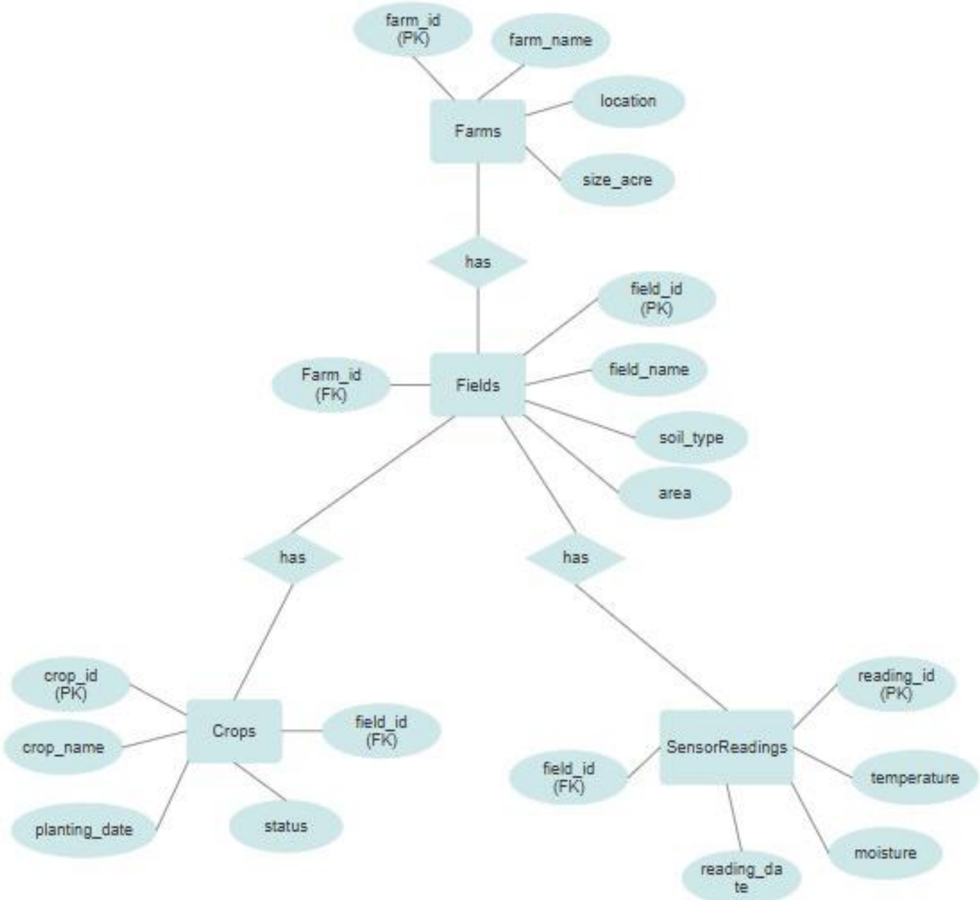
Introduction

This project presents a Smart Farming Database Management System designed to manage farms, fields, crops, and sensor data such as temperature and moisture. The system allows users to store, update, and analyze data through a user-friendly interface.

It assumes that each farm contains multiple fields, and each field has crops and sensor readings. The system helps monitor conditions and detect risks, supporting better decision-making in agriculture.

1) ER DIAGRAM





2) Data Definition Language Details (DDL)

Tables

Untitled | Save | Primary • Active | neondb

```sql
-- ========= TABLES =========

CREATE TABLE Roles (
    role_id SERIAL PRIMARY KEY,
    role_name TEXT NOT NULL
);

CREATE TABLE Users (
    user_id SERIAL PRIMARY KEY,
    username TEXT NOT NULL,
    password TEXT NOT NULL,
    full_name TEXT NOT NULL,
    role_id INT,
    FOREIGN KEY (role_id)
        REFERENCES Roles(role_id)
        ON DELETE SET NULL
);

CREATE TABLE Farms (
    farm_id SERIAL PRIMARY KEY,
    farm_name TEXT NOT NULL,
    location TEXT NOT NULL,
    size_acre NUMERIC
);
```

Untitled | Save | Primary • Active | neondb

```sql
CREATE TABLE Fields (
    field_id SERIAL PRIMARY KEY,
    farm_id INT,
    field_name TEXT NOT NULL,
    soil_type TEXT,
    area NUMERIC,
    FOREIGN KEY (farm_id)
        REFERENCES Farms(farm_id)
        ON DELETE CASCADE
);

CREATE TABLE Crops (
    crop_id SERIAL PRIMARY KEY,
    field_id INT,
    crop_name TEXT NOT NULL,
    planting_date DATE,
    status TEXT,
    FOREIGN KEY (field_id)
        REFERENCES Fields(field_id)
        ON DELETE CASCADE
);

CREATE TABLE SensorReadings (
    reading_id SERIAL PRIMARY KEY,
    field_id INT,
    temperature INT,
    moisture INT,
    reading_date TIMESTAMP,
    FOREIGN KEY (field_id)
        REFERENCES Fields(field_id)
        ON DELETE CASCADE
);
```

Connected (12 queries)

Run | Explain | Analyze

- crops
- farms
- fields
- roles
- sensorreadings
- users

Values

Untitled Save Primary Active neondb

```sql
);

-- ========= DATA =========

INSERT INTO Roles (role_name) VALUES
('Admin'),
('Technician'),
('Farmer');

INSERT INTO Users (username, password, full_name, role_id) VALUES
('admin', '1234', 'System Admin', 1),
('tech1', '1234', 'Hydroponic Technician', 2),
('farmer1', '1234', 'Strawberry Grower', 3);

INSERT INTO Farms (farm_name, location, size_acre) VALUES
('Hydro Strawberry Farm', 'Nicosia', 12),
('Smart Berry Lab', 'Kyrenia', 8);

INSERT INTO Fields (farm_id, field_name, soil_type, area) VALUES
(1, 'Greenhouse A', 'Hydroponic', 3),
(1, 'Greenhouse B', 'Hydroponic', 4),
(2, 'Vertical Tower 1', 'Hydroponic', 2);

INSERT INTO Crops (field_id, crop_name, planting_date, status) VALUES
(1, 'Strawberry Albion', '2026-03-01', 'Growing'),
(2, 'Strawberry Festival', '2026-02-15', 'Ready'),
(3, 'Strawberry San Andreas', '2026-03-10', 'Planted');

INSERT INTO SensorReadings (field_id, temperature, moisture, reading_date) VALUES
(1, 22, 70, '2026-05-06 10:00:00'),
(2, 24, 65, '2026-05-06 10:05:00'),
(3, 21, 75, '2026-05-06 10:10:00');
```

Ready to connect

Run

Untitled Save Primary Active neondb

```sql
INSERT INTO farms(farm_name) VALUES ('Hydro Strawberry Farm');
```

Connected (1 query)

Run Explain Analyze

ERROR: Farm name already exists! (SQLSTATE P0001) Fix with AI

#	field_id	farm_id	field_name	soil_type	area
1	1	1	Greenhouse A	Hydroponic	3
2	2	1	Greenhouse B	Hydroponic	4
3	3	2	Vertical Tower 1	Hydroponic	2

#	crop_id	field_id	crop_name	planting_date	status
1	1	1	Strawberry Albion	2026-03-01	Growing
2	2	2	Strawberry Festival	2026-02-15	Ready
3	3	3	Strawberry San Andreas	2026-03-10	Planted

#	farm_id	farm_name	location	size_acre
1	1	Hydro Strawberry Farm	Nicosia	12
2	2	Smart Berry Lab	Kyrenia	8

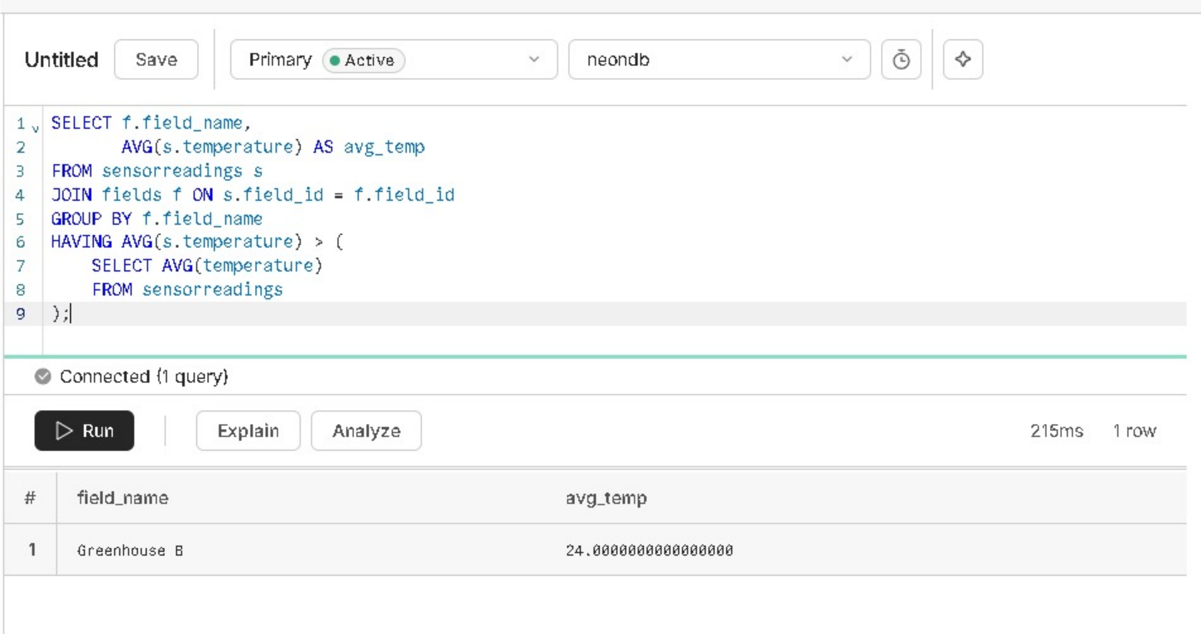
#	role_id	role_name
1	1	Admin
2	2	Technician
3	3	Farmer

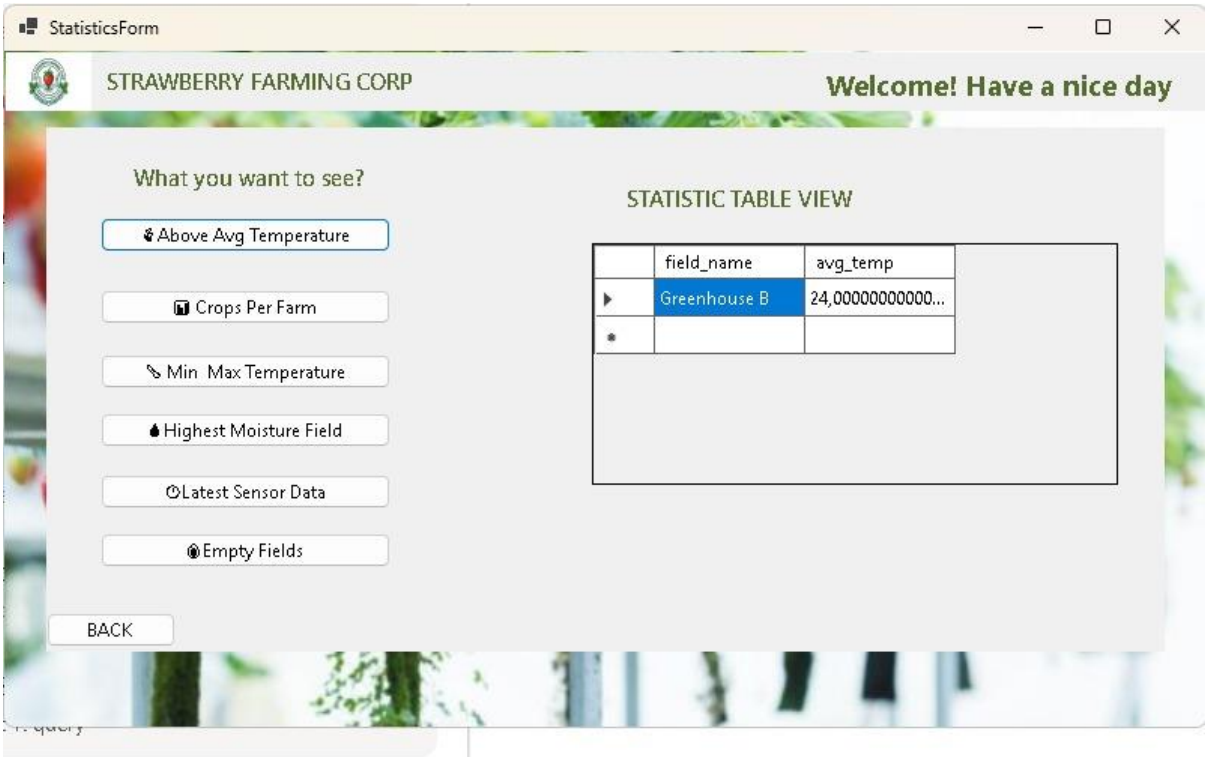
#	reading_id	field_id	temperature	moisture	reading_date
1	1	1	22	70	2026-05-06 10:00:00
2	2	2	24	65	2026-05-06 10:05:00
3	3	3	21	75	2026-05-06 10:10:00

#	user_id	username	password	full_name	role_id
1	1	admin	1234	System Admin	1
2	2	tech1	1234	Hydroponic Technician	2
3	3	farmer1	1234	Strawberry Grower	3

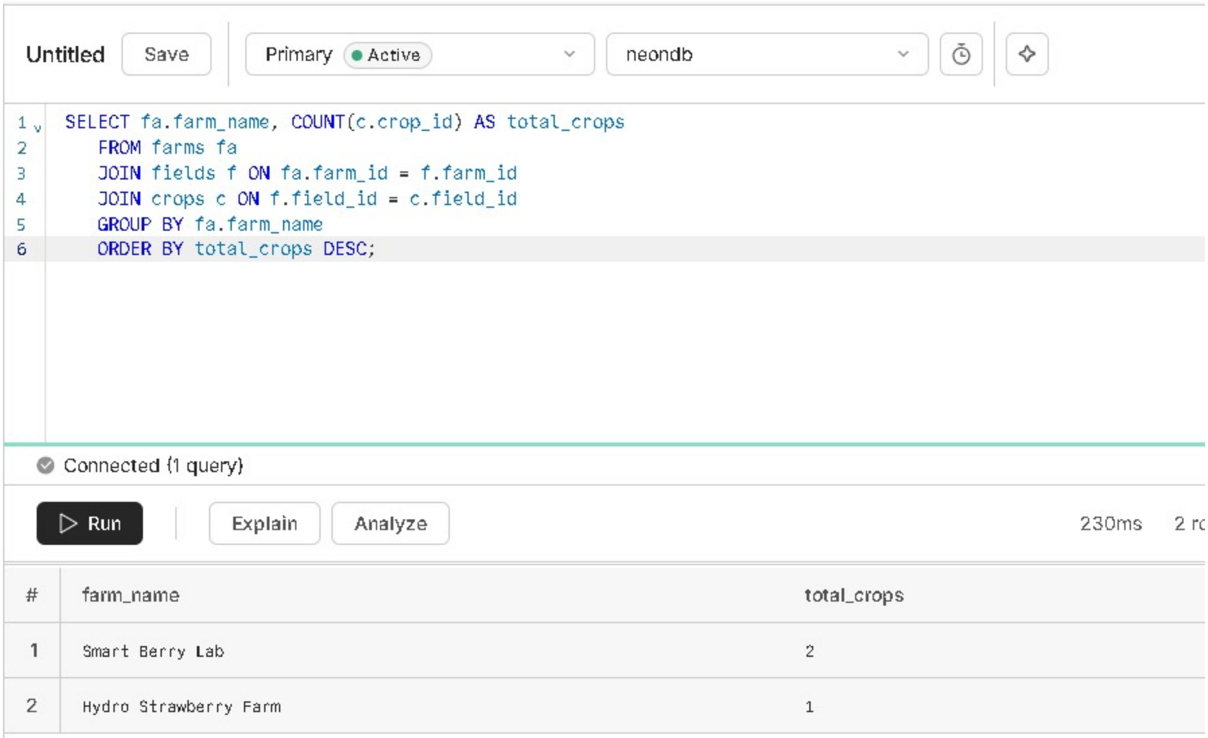
3) Data Manipulation Language (DML)

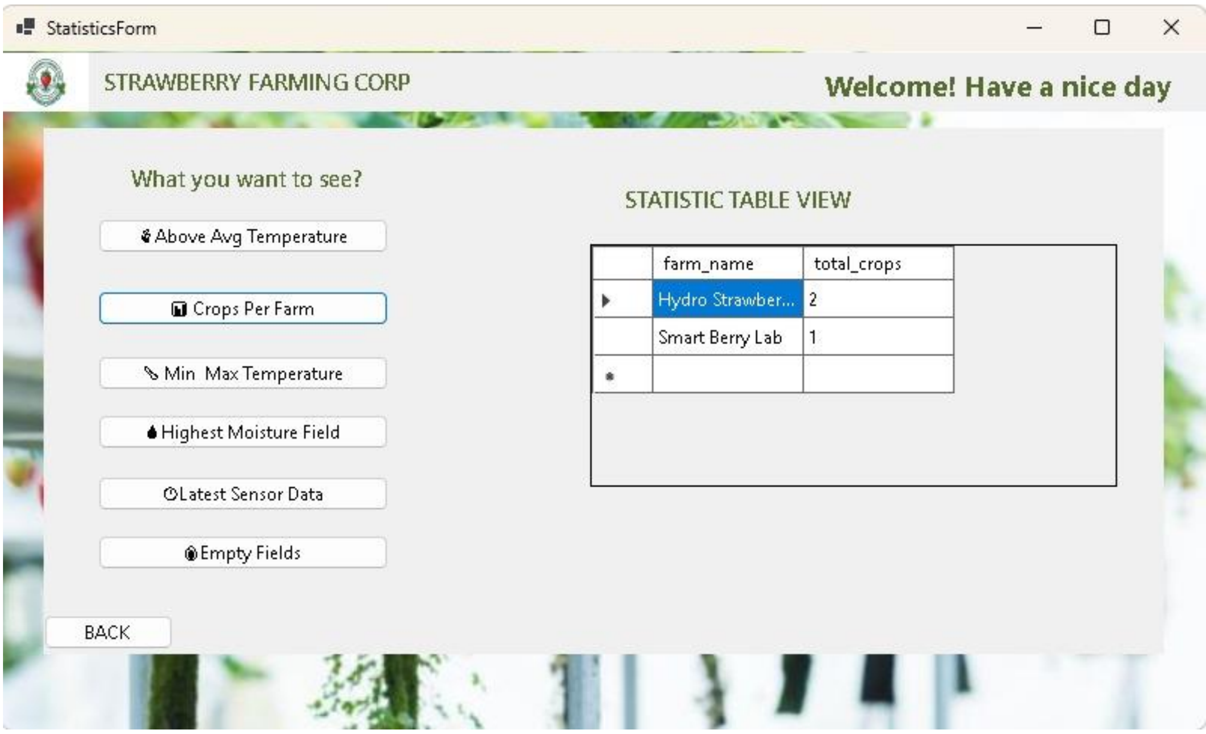
1.Query: This query finds fields whose average temperature is higher than the overall average temperature of all sensor readings.



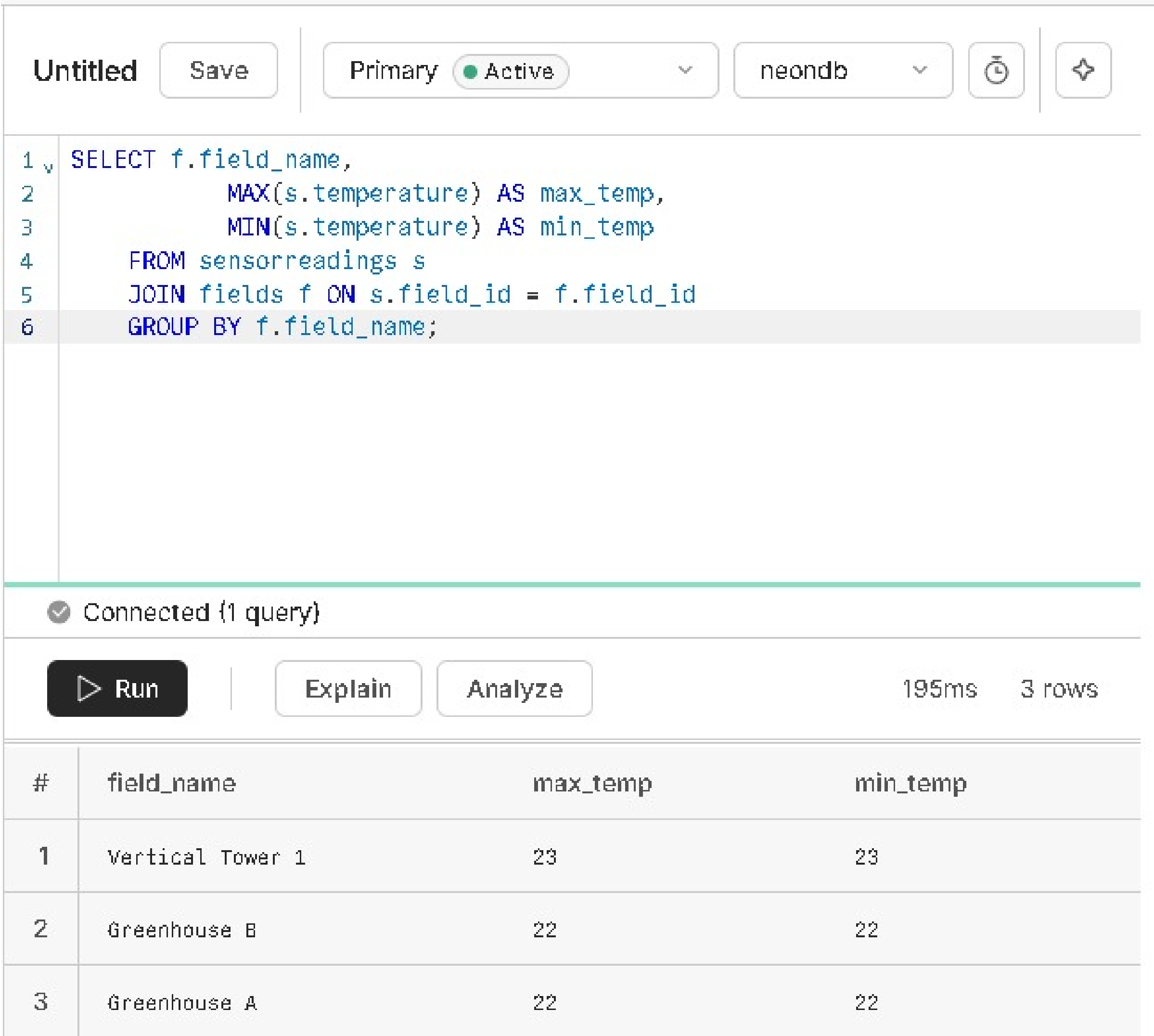


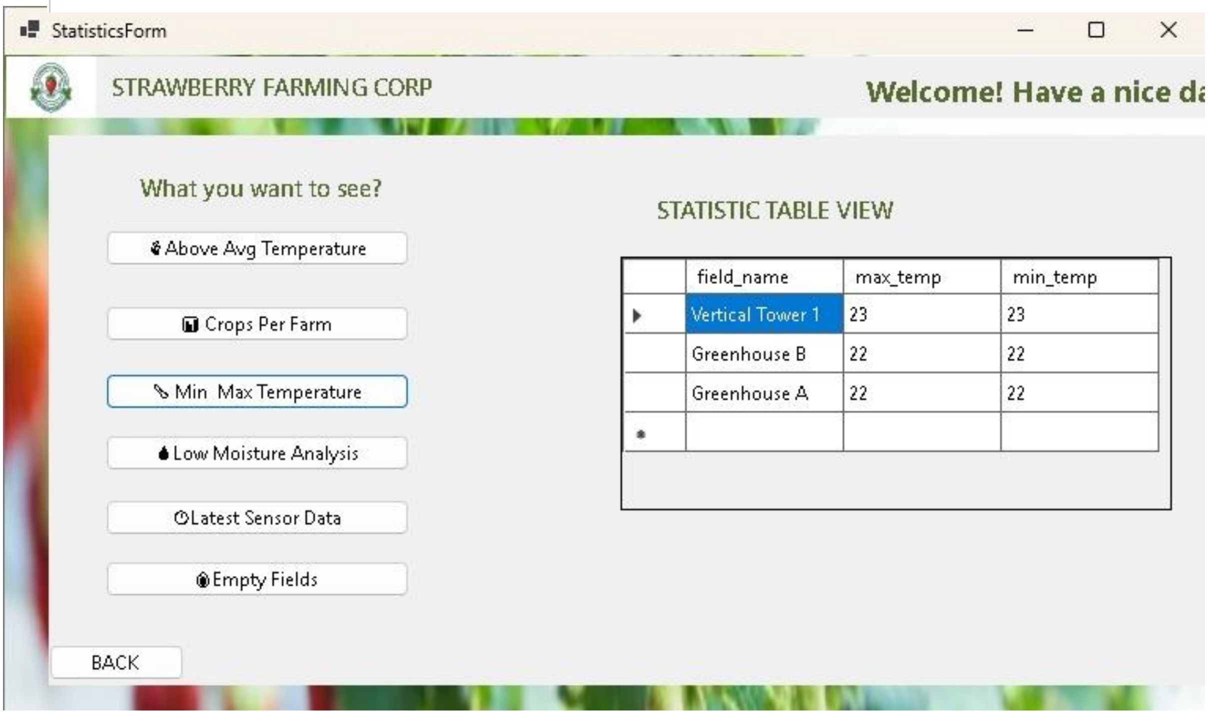
2.Query: This query calculates the total number of crops in each farm by joining farms, fields, and crops tables.



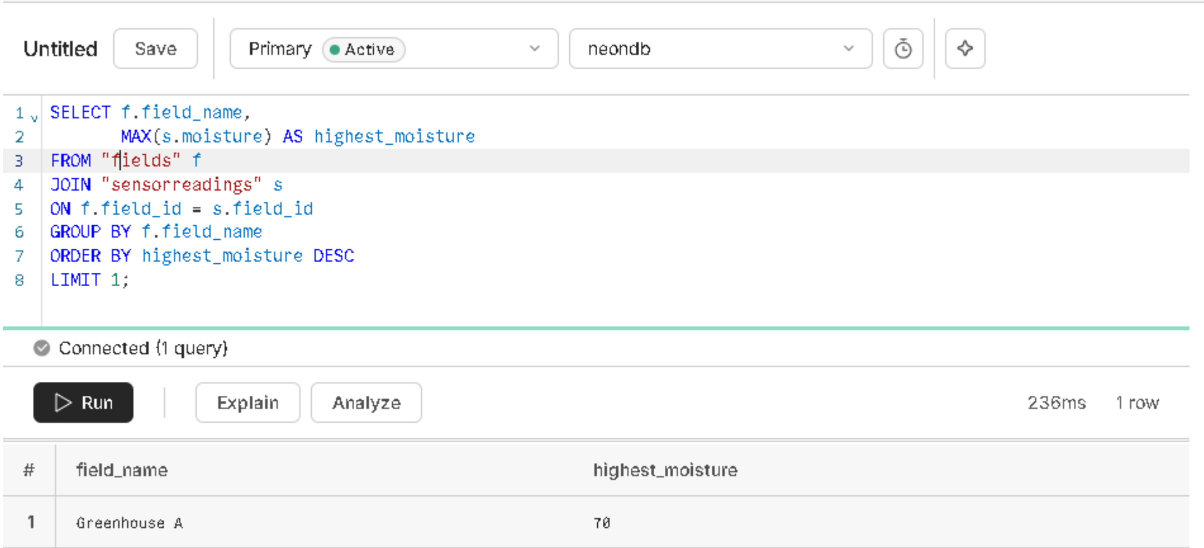


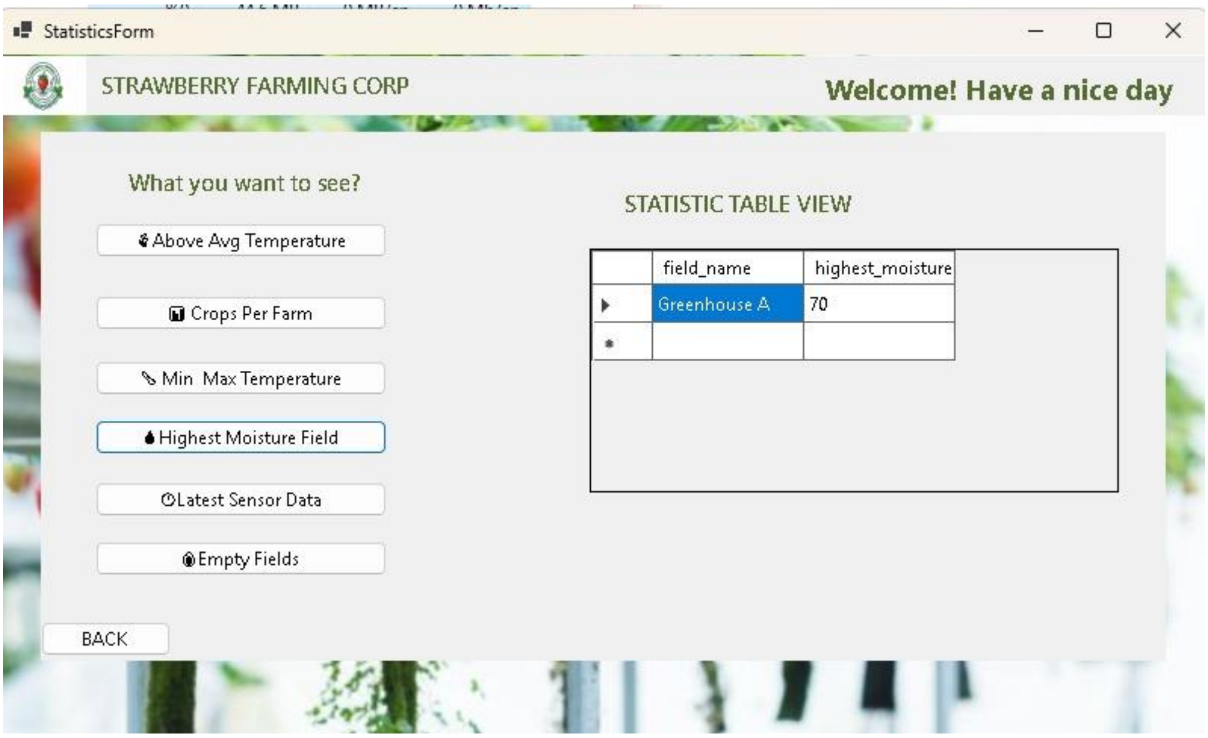
3.Query: This query shows the minimum and maximum temperature recorded in each field.



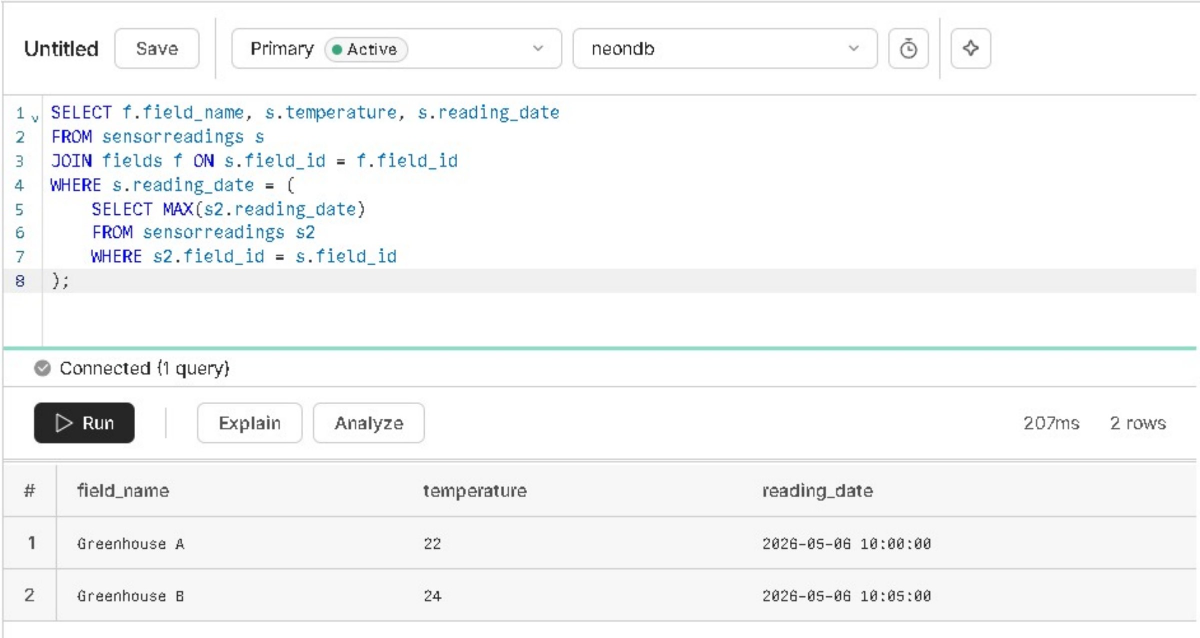


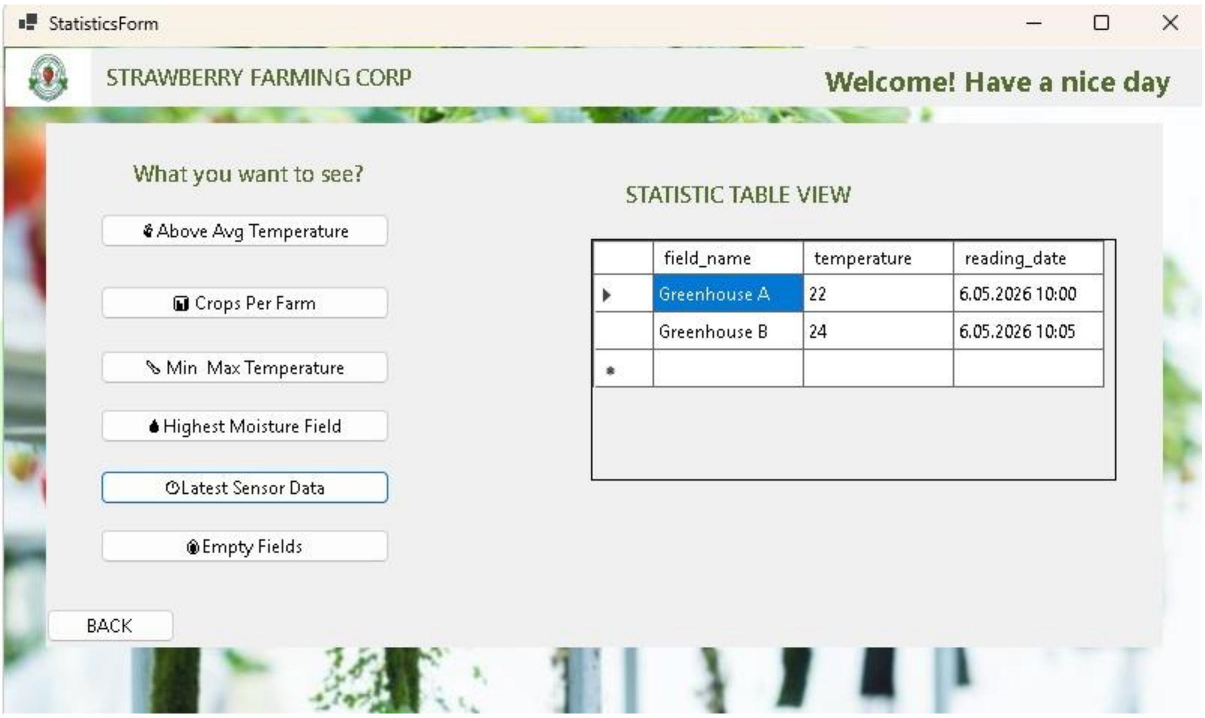
4.Query: This query finds the field with the highest moisture value by calculating the maximum moisture per field and returning the top result.



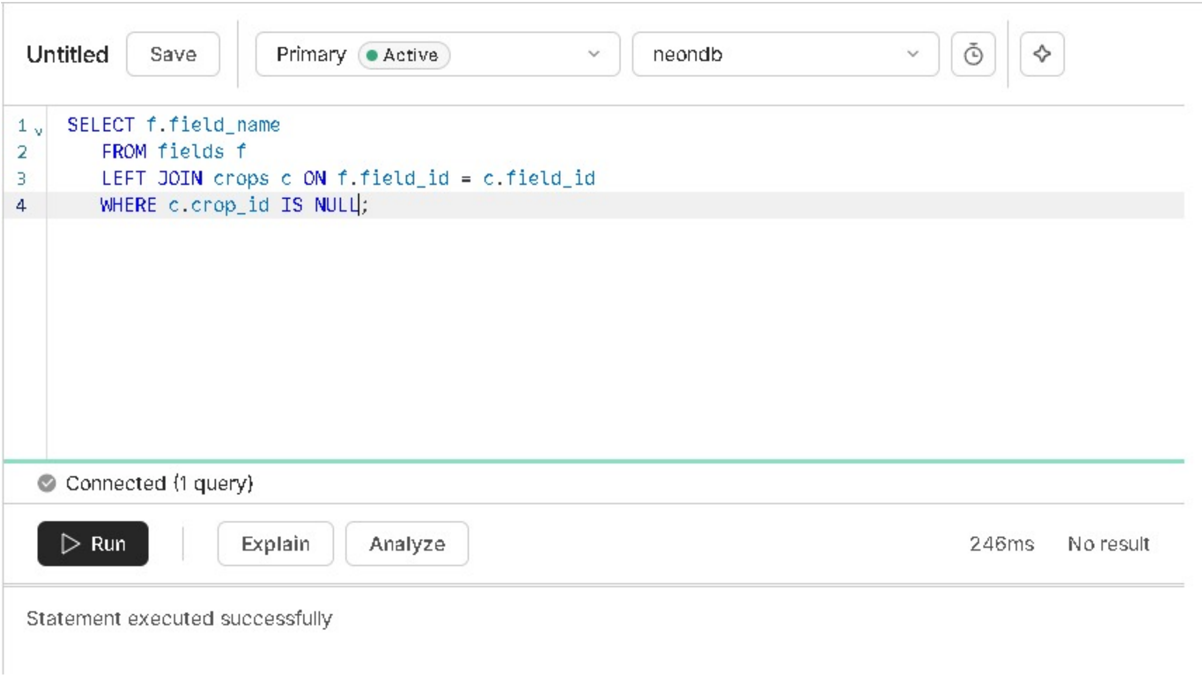


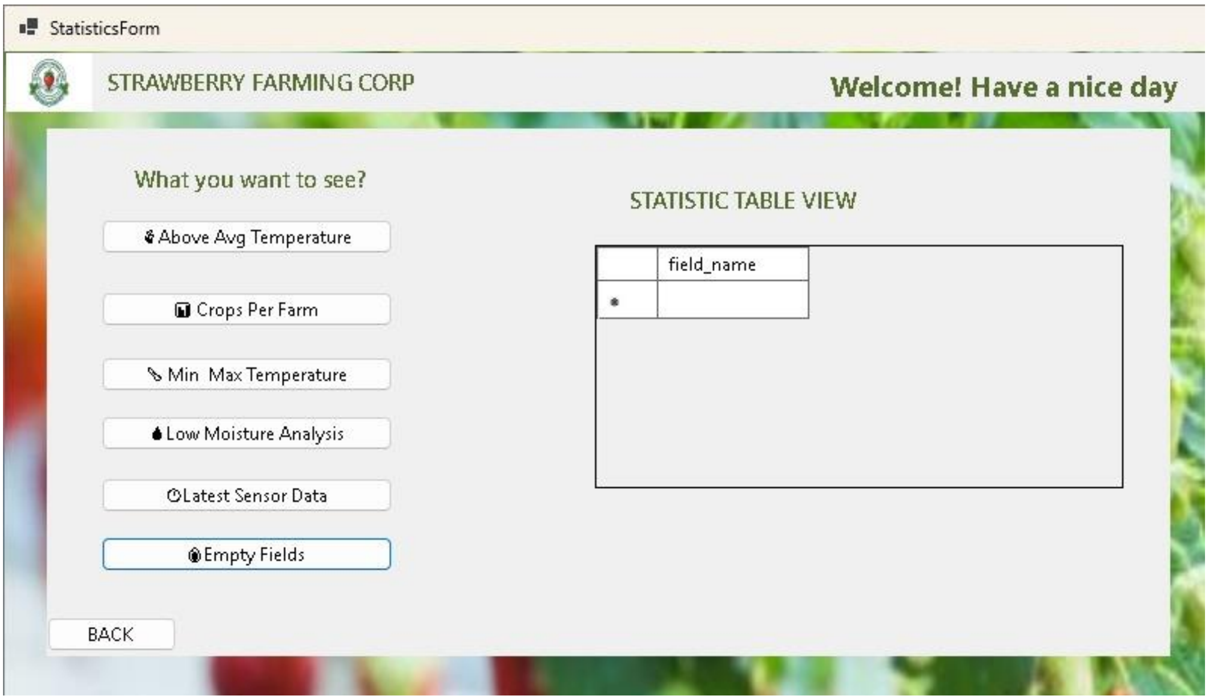
5.Query: This query retrieves the most recent temperature reading for each field by selecting records with the latest reading date from the sensorreadings table.



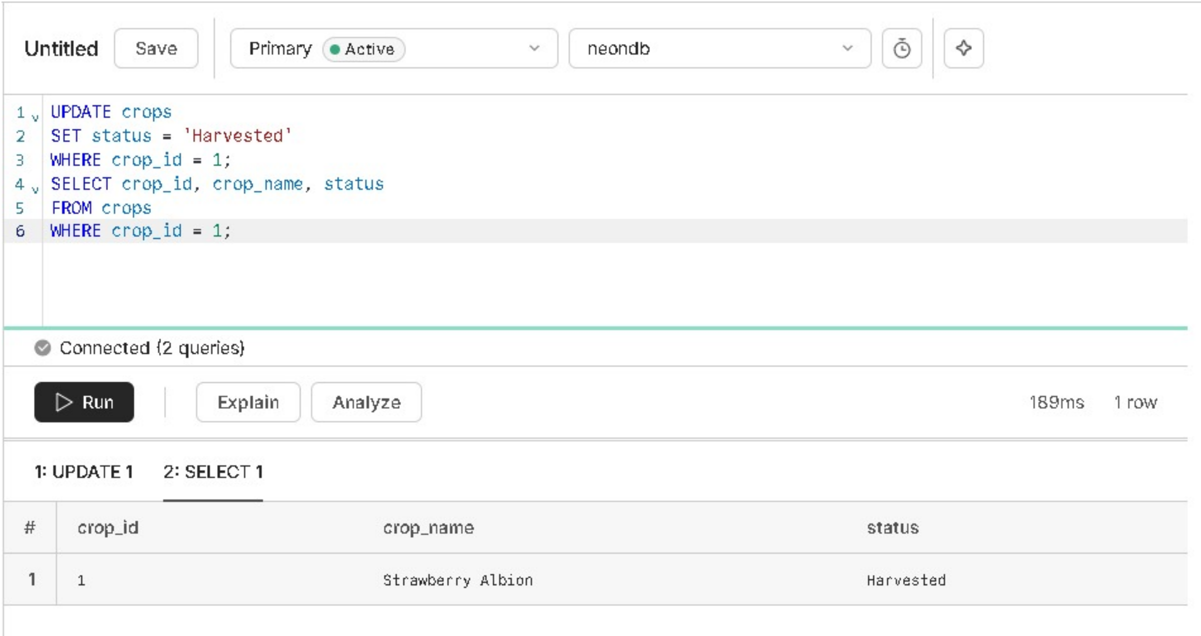


6.Query: This query lists all fields that do not currently contain any crops.

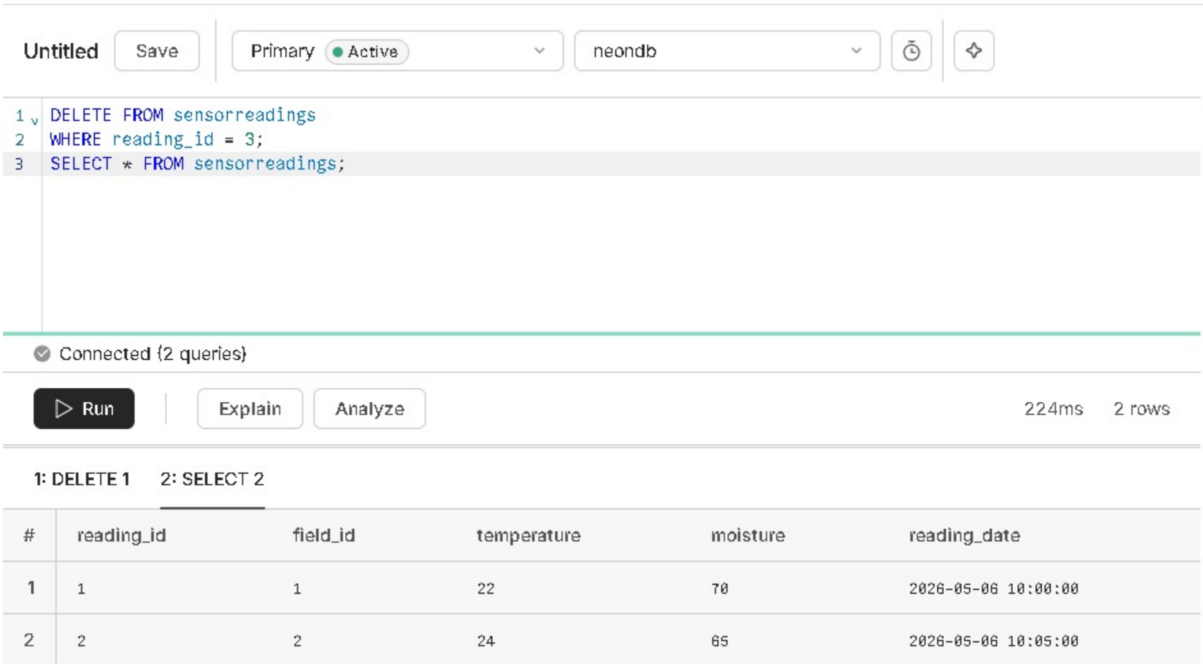

Untitled
Save
Primary
Active
neondb
SELECT f.field_name
FROM fields f
LEFT JOIN crops c ON f.field_id = c.field_id
WHERE c.crop_id IS NULL;
Connected (1 query)
Run
Explain
Analyze
246ms
No result
Statement executed successfully


StatisticsForm
STRAWBERRY FARMING CORP
Welcome! Have a nice day
What you want to see?
Above Avg Temperature
Crops Per Farm
Min Max Temperature
Low Moisture Analysis
Latest Sensor Data
Empty Fields
BACK
STATISTIC TABLE VIEW
field_name

Update (extra) Query:This query updates the status of a specific crop to "Harvested" based on its ID. It simulates a real-world scenario where a crop has completed its growth cycle.



Delete(extra) Query: This query deletes a specific sensor reading from the system using its ID. It is useful for removing incorrect or outdated data.



3) PL/SQL QUERIES

1,This procedure inserts a new crop into the database while preventing duplicate crop entries in the same field.

Untitled Save Primary Active neondb

```sql
CREATE OR REPLACE PROCEDURE add_crop(
    p_field_id INT,
    p_crop_name TEXT,
    p_date DATE
)
LANGUAGE plpgsql
AS $$
BEGIN
    IF NOT EXISTS (
        SELECT 1 FROM crops
        WHERE field_id = p_field_id AND crop_name = p_crop_name
    ) THEN
        INSERT INTO crops(field_id, crop_name, planting_date, status)
        VALUES (p_field_id, p_crop_name, p_date, 'New');
    ELSE
        RAISE NOTICE 'Crop already exists in this field';
    END IF;
END;
$$;
CALL add_crop(1, 'StrawberryX', '2026-05-14');
```

Connected (2 queries)

Run Explain Analyze 275ms No result

1: CREATE 2: CALL

Statement executed successfully

Untitled Save Primary Active neondb

```sql
SELECT crop_name, field_id FROM crops;
```

Connected (1 query)

Run Explain Analyze 197ms 1 row

#	crop_name	field_id
1	Strawberry Festival	2

2,This procedure automatically updates a field's crop status in the database to a specific risk level or "Healthy" by evaluating incoming temperature and moisture sensor data.

Untitled Save Primary Idle neondb

```sql
CREATE OR REPLACE PROCEDURE check_sensor_status(
    p_field_id BIGINT,
    p_temperature NUMERIC,
    p_moisture NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
    IF p_temperature > 30 THEN
        UPDATE crops
        SET status = 'High Temperature Risk'
        WHERE field_id = p_field_id;

        RAISE NOTICE 'Temperature is too high. Crop status updated.';
    ELSIF p_moisture < 40 THEN
        UPDATE crops
        SET status = 'Low Moisture Risk'
        WHERE field_id = p_field_id;

        RAISE NOTICE 'Moisture is too low. Crop status updated.';
    ELSE
       UPDATE crops
        SET status = 'Healthy'
        WHERE field_id = p_field_id;

        RAISE NOTICE 'Field conditions are normal.';

    END IF;

END;
$$;
CALL check_sensor_status(1, 35, 50);
```

Connected (2 queries)

Run Explain Analyze 228ms No result

1: CRE

Temperature is too high. Crop status updated.

Statement executed successfully

3,This procedure inserts a new account record into the users table using the provided username, password, full name, and role ID.

Untitled Save Primary Active neondb

```sql
CREATE OR REPLACE PROCEDURE add_new_user(
    p_username TEXT,
    p_password TEXT,
    p_full_name TEXT,
    p_role_id BIGINT
)
LANGUAGE plpgsql
AS $$
BEGIN

    INSERT INTO users(
        username,
        password,
        full_name,
        role_id
    )
    VALUES (
        p_username,
        p_password,
        p_full_name,
        p_role_id
    );

    RAISE NOTICE 'New user added successfully.';

END;
$$;
CALL add_new_user('zehra', '1234', 'Zehra Tas', 1);
```

Connected (2 queries)

Run Explain Analyze 197ms No result

1: CREATE 2: CALL

Statement executed successfully

New user added successfully.

4,This function returns the total number of crops in a given field for analytical purposes.

Untitled Save Primary Active neondb

```sql
CREATE OR REPLACE FUNCTION get_crop_count(p_field_id INT)
RETURNS INT
LANGUAGE plpgsql
AS $$
DECLARE total INT;
BEGIN
    SELECT COUNT(*) INTO total
    FROM crops
    WHERE field_id = p_field_id;

    RETURN total;
END;
$$;
SELECT get_crop_count(1);
```

Connected (2 queries)

Run Explain Analyze 205ms 1 row

1: CREATE 2: SELECT 1

#	get_crop_count
1	0

5,TRIGGER Detects low moisture levels and warns the system to prevent crop damage.

Untitled Save Primary Active neondb

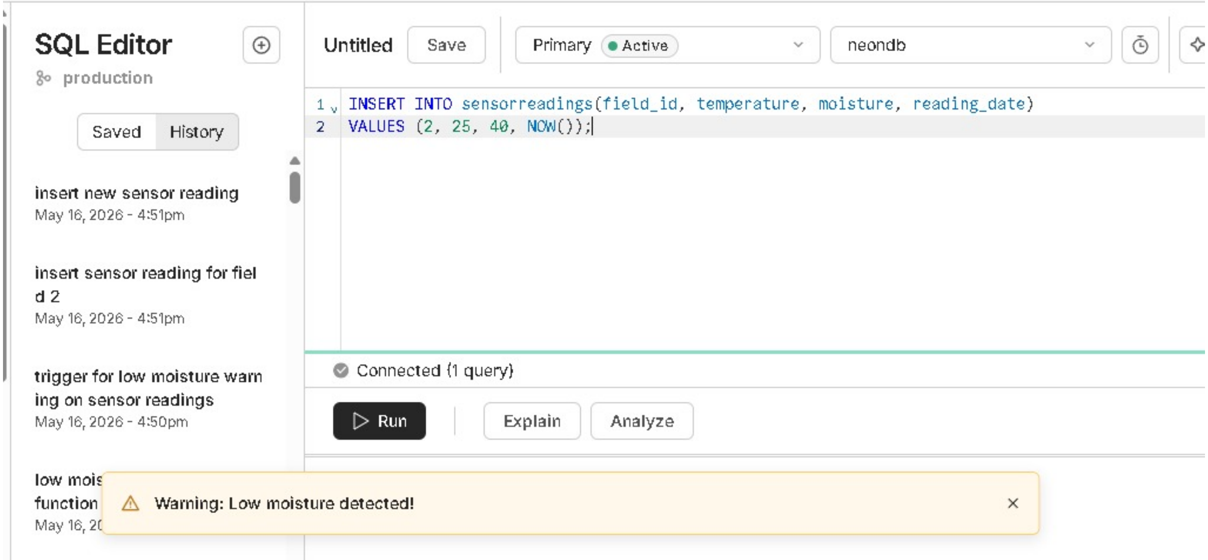
```sql
CREATE OR REPLACE FUNCTION low_moisture_warning()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.moisture < 60 THEN
        RAISE NOTICE 'Warning: Low moisture detected!';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_low_moisture
AFTER INSERT ON sensorreadings
FOR EACH ROW
EXECUTE FUNCTION low_moisture_warning();
```

Connected (2 queries)

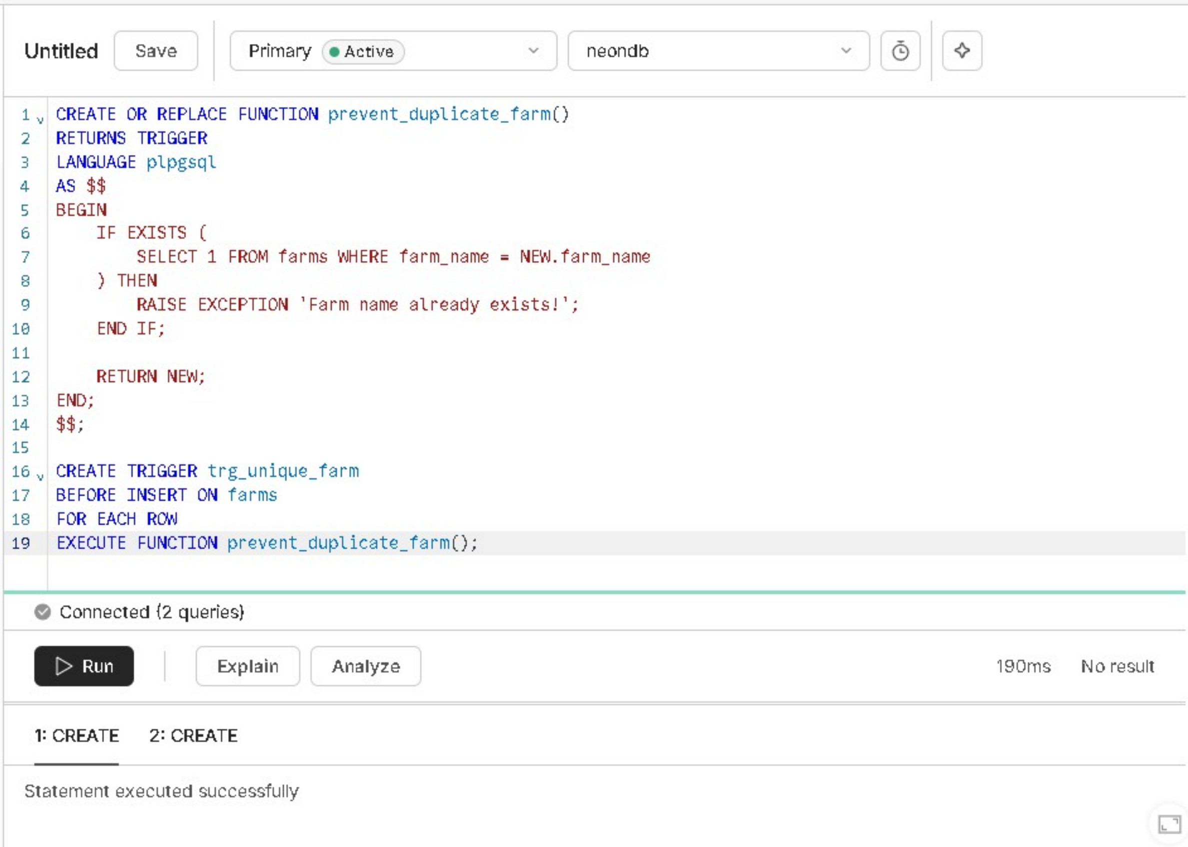
Run Explain Analyze 201ms No result

1: CREATE 2: CREATE

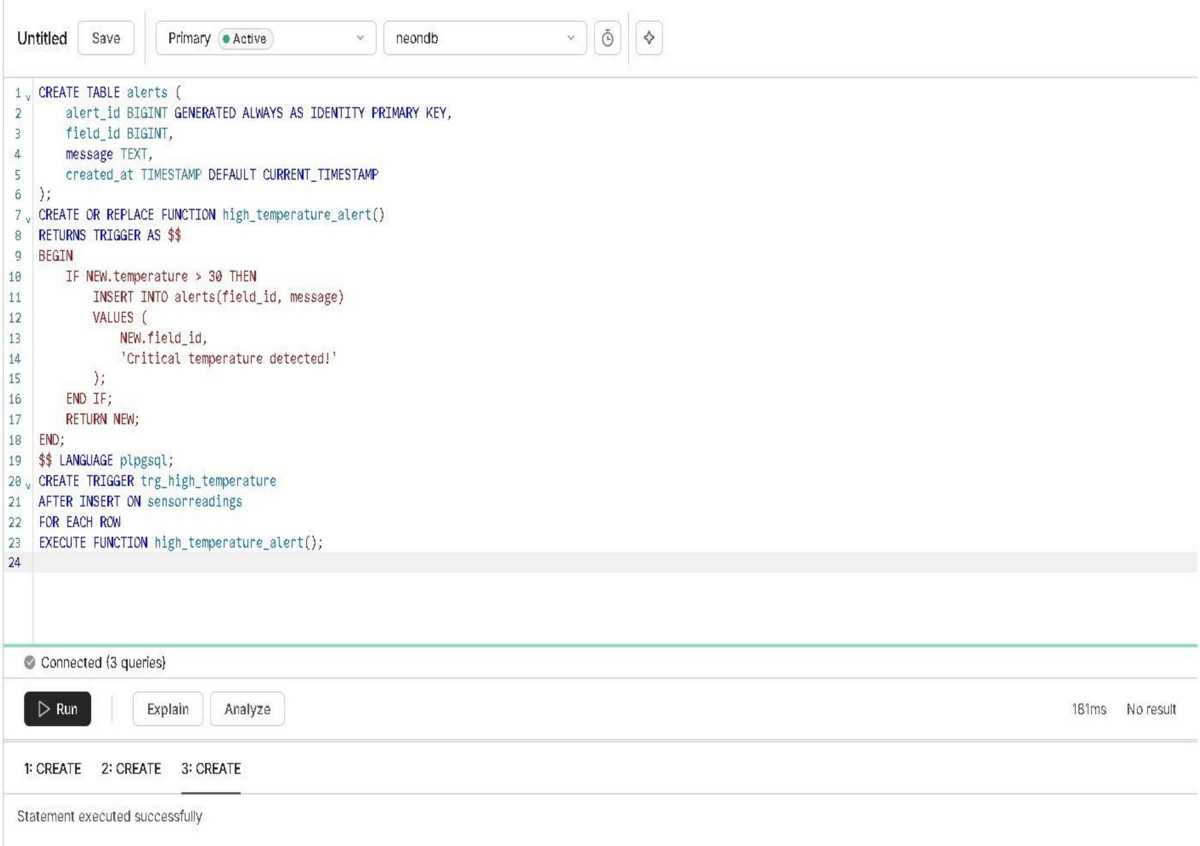
Statement executed successfully



6,This trigger prevents insertion of farms with duplicate names to maintain data integrity.

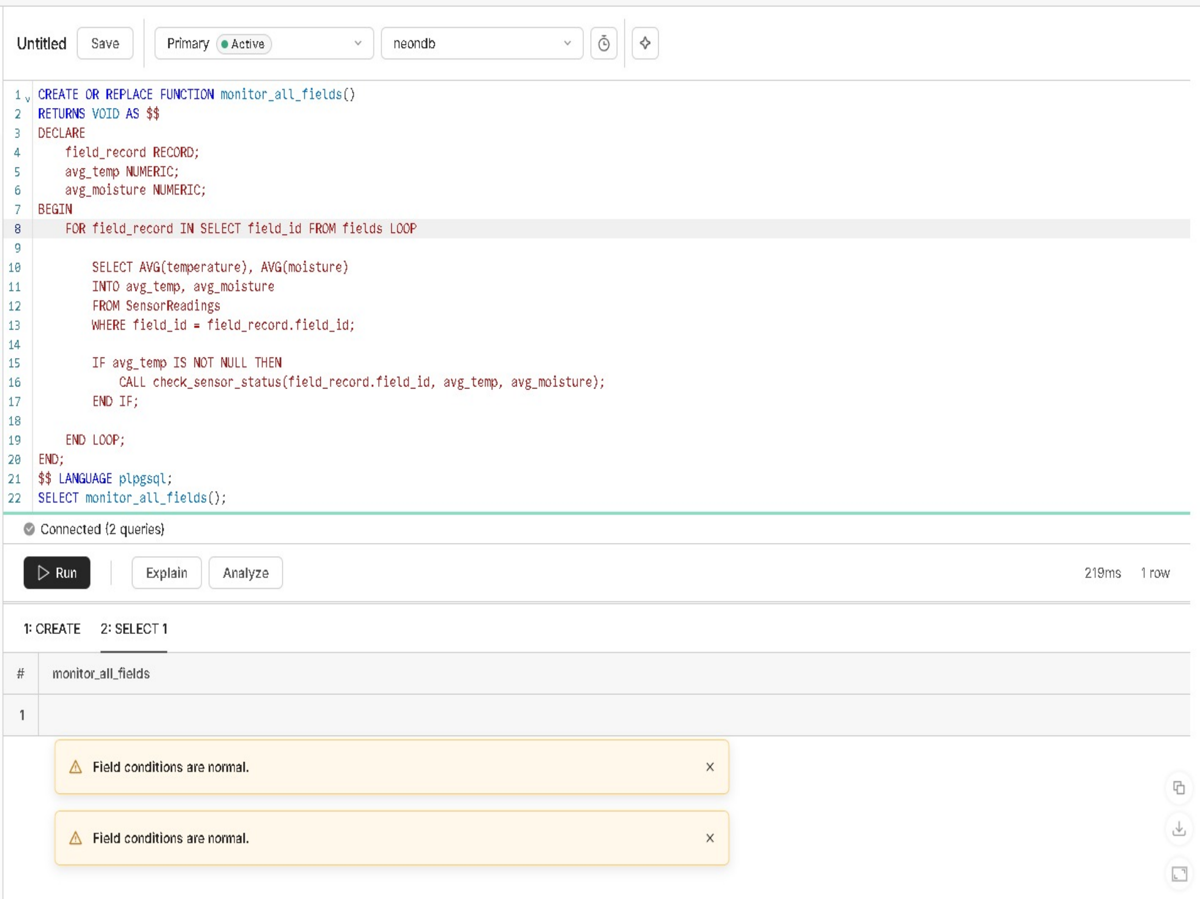


7,This trigger logs a "Critical temperature detected!" message into an alerts table whenever a new sensor reading exceeds 30 degrees.




```sql
CREATE TABLE alerts (
    alert_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    field_id BIGINT,
    message TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE OR REPLACE FUNCTION high_temperature_alert()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.temperature > 30 THEN
        INSERT INTO alerts(field_id, message)
        VALUES (
            NEW.field_id,
            'Critical temperature detected!'
        );
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_high_temperature
AFTER INSERT ON sensorreadings
FOR EACH ROW
EXECUTE FUNCTION high_temperature_alert();
```

8,This PL/pgSQL function iterates through every field in the database, calculates their average temperature and moisture levels from recent sensor readings, and executes a status check procedure to evaluate and update their crop conditions.



4)DB Connection

This section explains how the application connects to the PostgreSQL database using **Npgsql**.
The connection is established using a connection string obtained from the cloud database provider.
A simple SQL query is executed to verify user login credentials (username and password).

```
        1 başvuru
        private void btnLogin_Click(object sender, EventArgs e)
        {
            using (var conn = new NpgsqlConnection(connString))
            {
                conn.Open();

                string query = @"SELECT * FROM Users
                        WHERE username=@u AND password=@p";
```

```
namespace SmartFarmingSystem
{
    6 başvuru
    public partial class LoginForm : Form
    {
        string connString = "Host=ep-bold-surf-apre6yz3.c-7.us-east-1.aws.neon.tech;Database=neondb;Username=neondb_owner;Password=npg_nqxUsDFfP10g;SSL Mode=Require;Trust Server Certificate=true;";
```

Connect your app manually

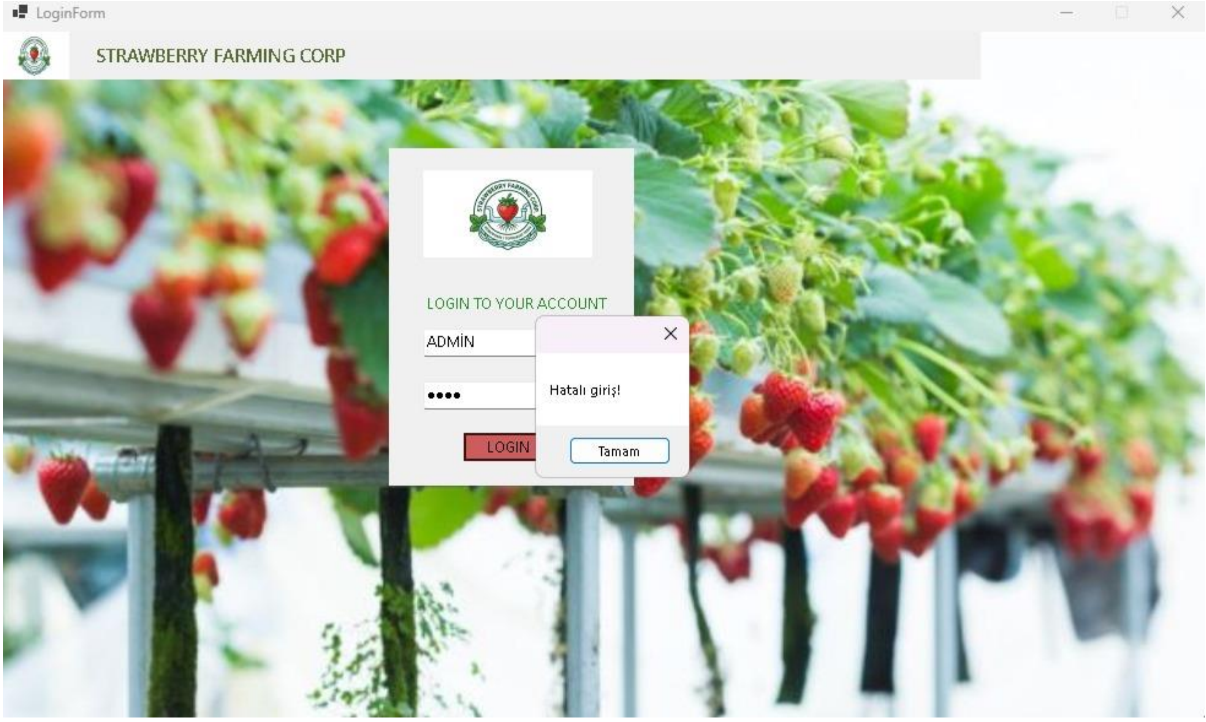
Copy and paste your production branch's connection details into your app's .env file

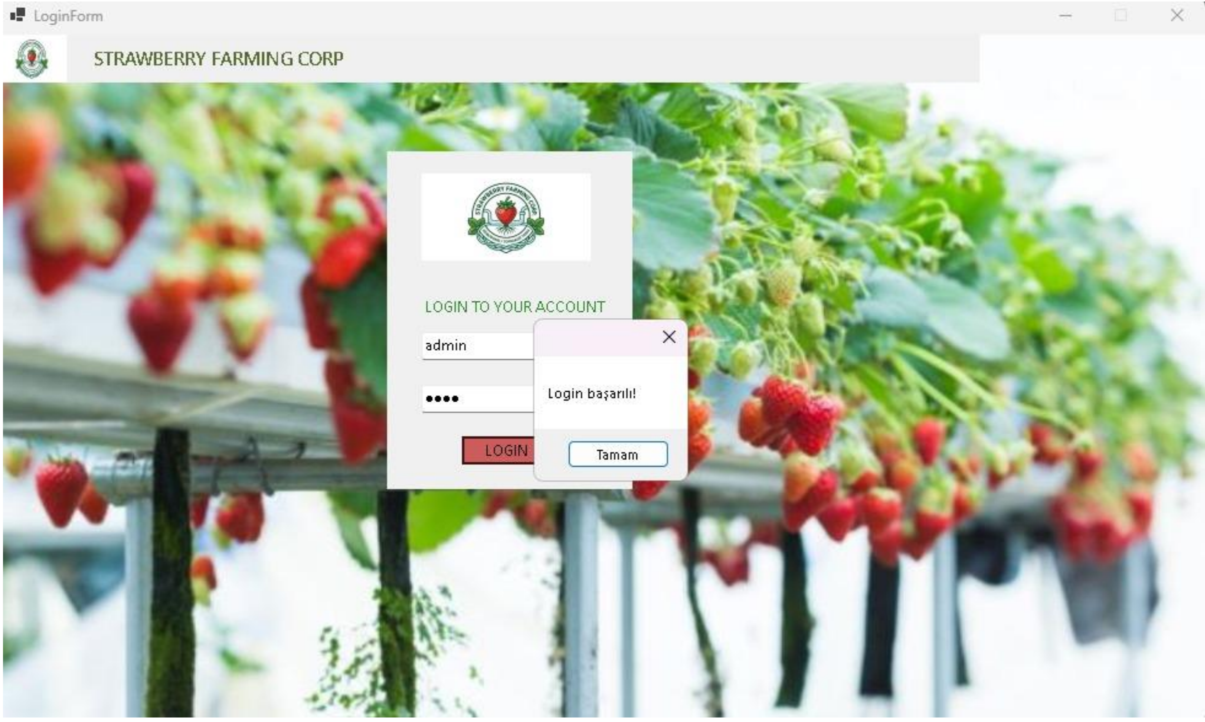
Connection string | Connection parameters

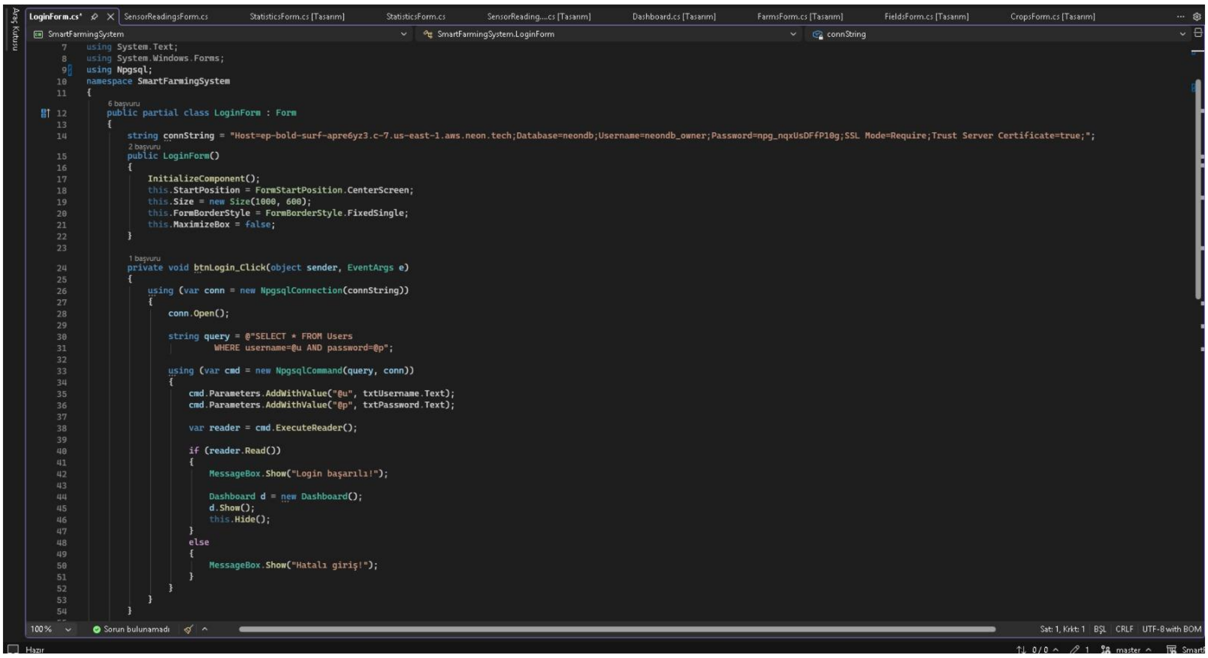
Host	ep-bold-surf-apre6yz3.c-7.us-east-1.aws.neon.tech
Database	neondb
Role	neondb_owner
Password	************
Pooler host	ep-bold-surf-apre6yz3-pooler.c-7.us-east-1.aws.neon.tech

5)Codes and GUI

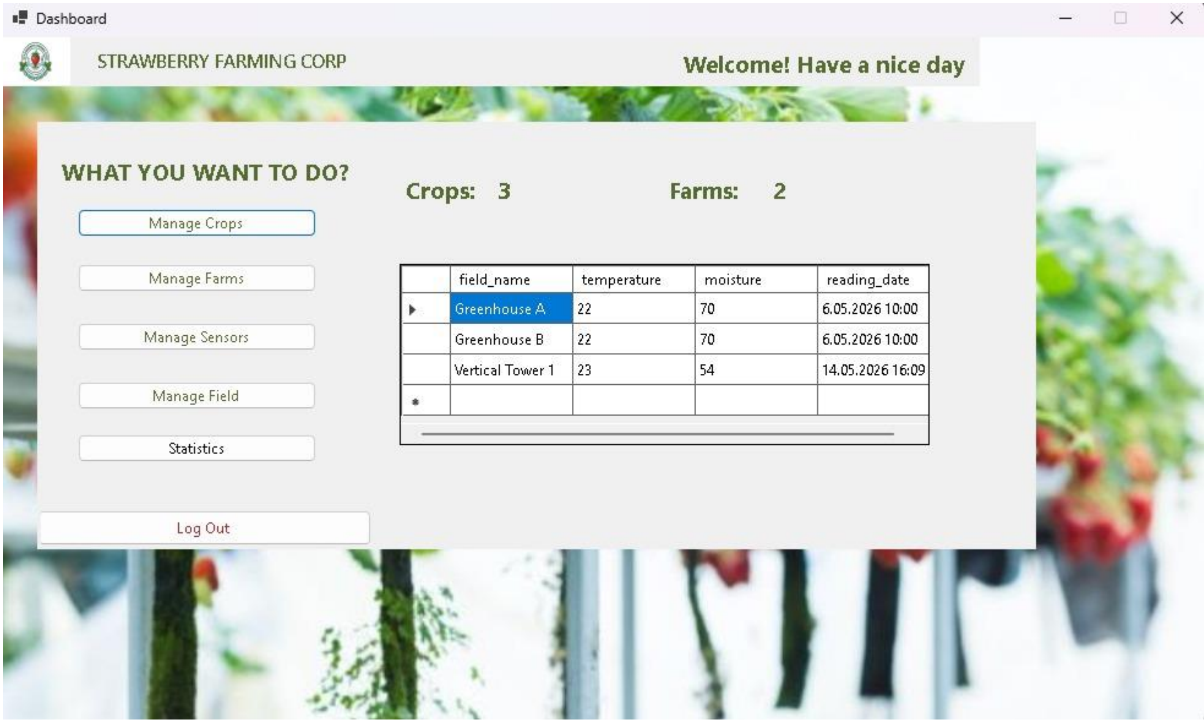
Login Page


LoginForm
STRAWBERRY FARMING CORP
LOGIN TO YOUR ACCOUNT
ADMİN
••••
LOGIN
Hatalı giriş!
Tamam


LoginForm
STRAWBERRY FARMING CORP
LOGIN TO YOUR ACCOUNT
admin
••••
LOGIN
Login başarılı!
Tamam


LoginForm.cs*
SensorReadingsForm.cs
StatisticsForm.cs [Tasarım]
StatisticsForm.cs
SensorReading....cs [Tasarım]
Dashboard.cs [Tasarım]
FarmsForm.cs [Tasarım]
FieldsForm.cs [Tasarım]
CropsForm.cs [Tasarım]
SmartFarmingSystem
SmartFarmingSystem.LoginForm
connString
using System.Text;
using System.Windows.Forms;
using Npgsql;
namespace SmartFarmingSystem
{
6 başvuru
public partial class LoginForm : Form
{
string connString = "Host=ep-bold-surf-apre6yz3.c-7.us-east-1.aws.neon.tech;Database=neondb;Username=neondb_owner;Password=npg_nqxUsDFfP10g;SSL Mode=Require;Trust Server Certificate=true;";
2 başvuru
public LoginForm()
{
InitializeComponent();
this.StartPosition = FormStartPosition.CenterScreen;
this.Size = new Size(1000, 600);
this.FormBorderStyle = FormBorderStyle.FixedSingle;
this.MaximizeBox = false;
}
1 başvuru
private void btnLogin_Click(object sender, EventArgs e)
{
using (var conn = new NpgsqlConnection(connString))
{
conn.Open();
string query = @"SELECT * FROM Users
WHERE username=@u AND password=@p";
using (var cmd = new NpgsqlCommand(query, conn))
{
cmd.Parameters.AddWithValue("@u", txtUsername.Text);
cmd.Parameters.AddWithValue("@p", txtPassword.Text);
var reader = cmd.ExecuteReader();
if (reader.Read())
{
MessageBox.Show("Login başarılı!");
Dashboard d = new Dashboard();
d.Show();
this.Hide();
}
else
{
MessageBox.Show("Hatalı giriş!");
}
}
}
}
100 %
Sorun bulunamadı
Hazır

Dashboard

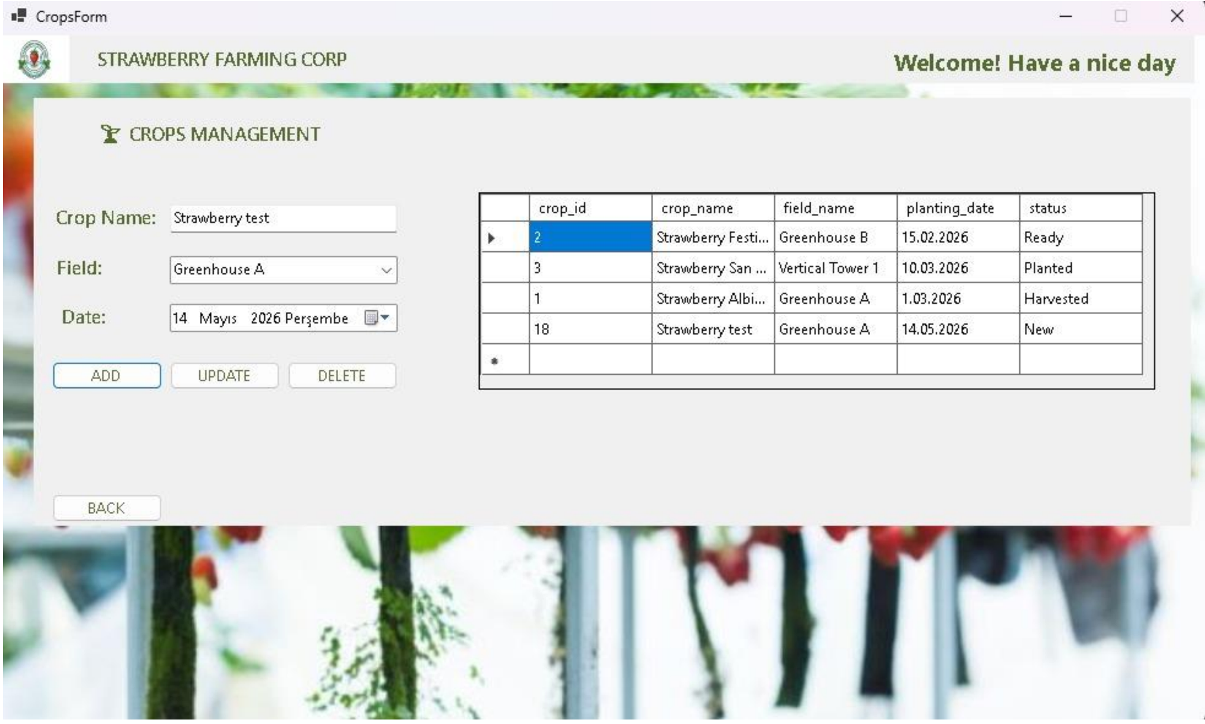

Dashboard
STRAWBERRY FARMING CORP
Welcome! Have a nice day
WHAT YOU WANT TO DO?
Crops: 3
Farms: 2
Manage Crops
Manage Farms
Manage Sensors
Manage Field
Statistics
Log Out
field_name | temperature | moisture | reading_date
Greenhouse A | 22 | 70 | 6.05.2026 10:00
Greenhouse B | 22 | 70 | 6.05.2026 10:00
Vertical Tower 1 | 23 | 54 | 14.05.2026 16:09

```csharp
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Text;
using System.Windows.Forms;
using Npgsql;

namespace SmartFarmingSystem
{
    using System.Data;
    public partial class Dashboard : Form
    {
        string connString = "Host=ep-bold-surf-apre6yz3.c-7.us-east-1.aws.neon.tech;Database=neondb;Username=neondb_owner;Password=npg_nqxUsDFfP10g;SSL Mode=Require;Trust Server Certificate=true;";



        public Dashboard()
        {
            InitializeComponent();
            this.StartPosition = FormStartPosition.CenterScreen;
            this.Size = new Size(1000, 600);
            this.FormBorderStyle = FormBorderStyle.FixedSingle;
            this.MaximizeBox = false;
        }


        private void Dashboard_Load(object sender, EventArgs e)
        {
            LoadDashboardData();
        }
        void LoadDashboardData()
        {
            using (var conn = new NpgsqlConnection(connString))
            {
                conn.Open();

                // toplam crops
                using (var cmd = new NpgsqlCommand("SELECT COUNT(*) FROM Crops", conn))
                {
                    lblCrops.Text = cmd.ExecuteScalar().ToString();
                }

                // toplam farms
                using (var cmd = new NpgsqlCommand("SELECT COUNT(*) FROM Farms", conn))
                {
                    lblFarms.Text = cmd.ExecuteScalar().ToString();
```

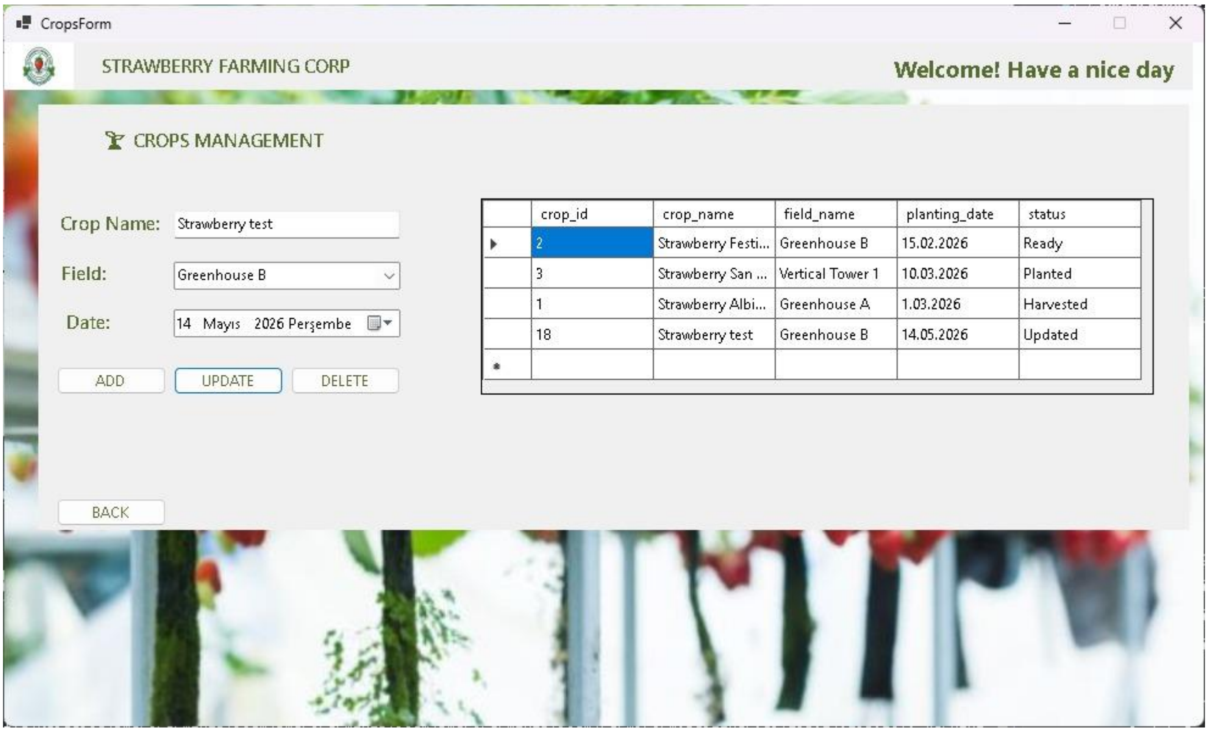
```csharp
                using (var cmd = new NpgsqlCommand("SELECT COUNT(*) FROM Farms", conn))
                {
                    lblFarms.Text = cmd.ExecuteScalar().ToString();
                }

                // sensör tablosu
                string query = @"SELECT f.field_name, s.temperature, s.moisture, s.reading_date
                         FROM SensorReadings s
                         JOIN Fields f ON s.field_id = f.field_id";

                using (var da = new NpgsqlDataAdapter(query, conn))
                {
                    DataTable dt = new DataTable();
                    da.Fill(dt);
                    dataGridViewtbl1.DataSource = dt;
                }
            }
        }

        private void label3_Click(object sender, EventArgs e)
        {

        }

        private void panel2_Paint(object sender, PaintEventArgs e)
        {

        }

        private void button2_Click(object sender, EventArgs e)
        {
            FarmsForm f = new FarmsForm(); // şimdilik bunu aç
            f.Show();
            this.Close();
        }

        private void button4_Click(object sender, EventArgs e)
        {
            FieldsForm f = new FieldsForm();
            f.Show();
            this.Close();
        }

```

Crops management

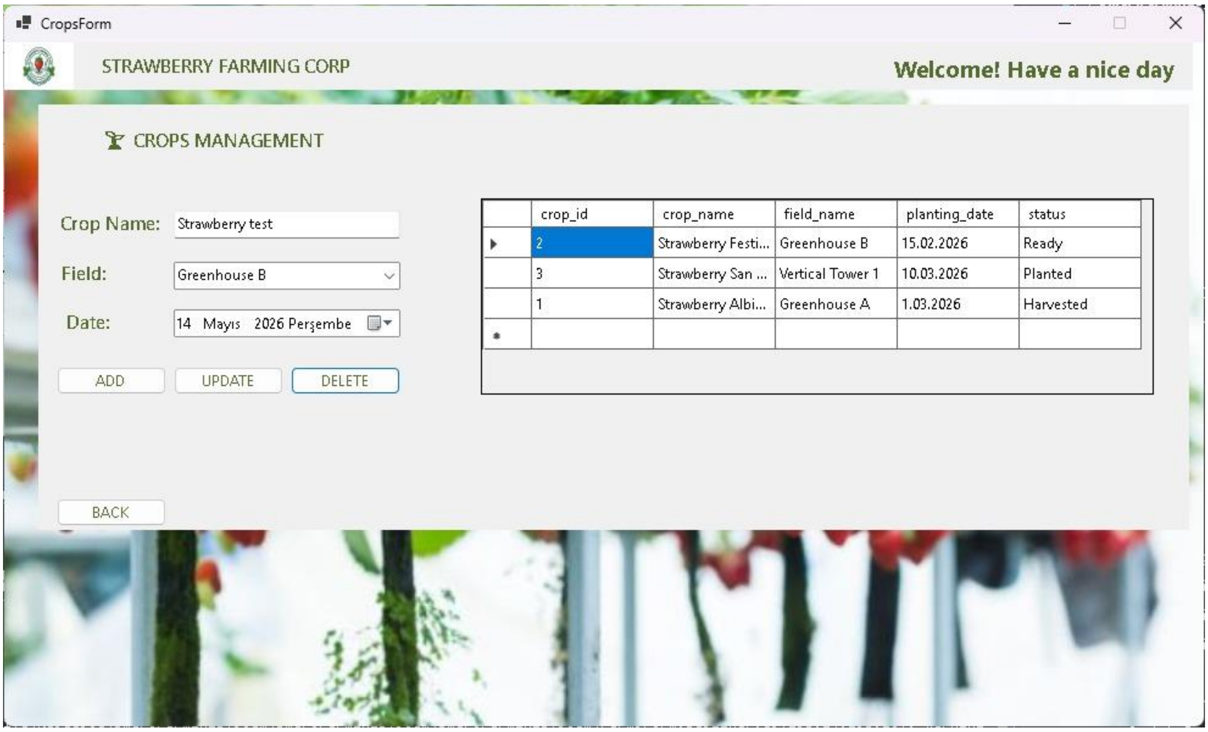
Adding crop



Updating crop



Deleting crop



CropsForm.cs FieldsForm.cs

SmartFarmingSystem | SmartFarmingSystem.CropsForm | connString

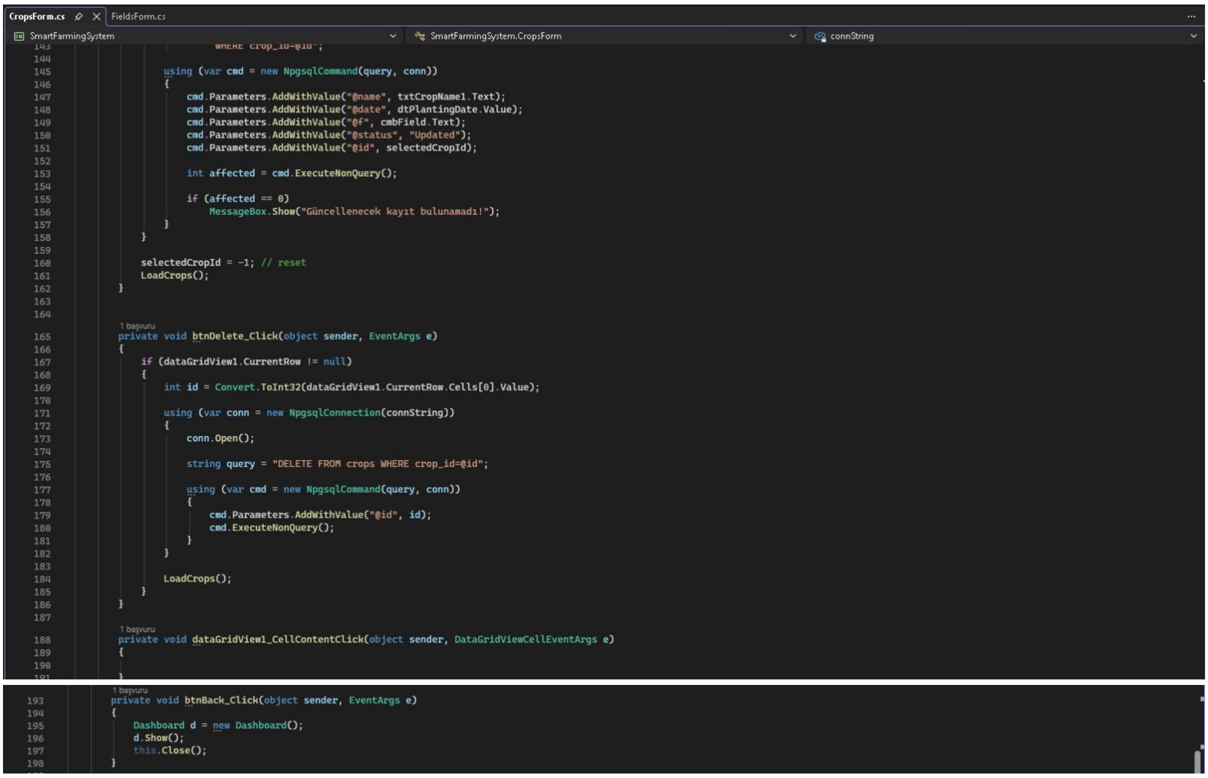
```csharp
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Text;
using System.Windows.Forms;
using Npgsql;

namespace SmartFarmingSystem
{
    public partial class CropsForm : Form
    {
        string connString = "Host=ep-bold-surf-apre6yz3.c-7.us-east-1.aws.neon.tech;Database=neondb;Username=neondb_owner;Password=npg_nqxUsDFfP10g;SSL Mode=Require;Trust Server Certificate=true;";
        int selectedCropId = -1;

        public CropsForm()
        {
            InitializeComponent();
            this.Load += CropsForm_Load;
            this.StartPosition = FormStartPosition.CenterScreen;
            this.Size = new Size(1000, 600);
            this.FormBorderStyle = FormBorderStyle.FixedSingle;
            this.MaximizeBox = false;
        }

        void LoadCrops()
        {
            using (var conn = new NpgsqlConnection(connString))
            {
                conn.Open();

                string query = @"SELECT c.crop_id, c.crop_name, f.field_name, c.planting_date, c.status
                        FROM crops c
                        JOIN fields f ON c.field_id = f.field_id";

                using (var da = new NpgsqlDataAdapter(query, conn))
                {
                    DataTable dt = new DataTable();
                    da.Fill(dt);
                    dataGridView1.DataSource = dt;
                }
            }
        }

        void LoadFields()
        {
            cmbField.Items.Clear();

            cmbField.Items.Add("Greenhouse A");
```

```
        cmbField.Items.Add("Greenhouse A");
        cmbField.Items.Add("Greenhouse B");
        cmbField.Items.Add("Vertical Tower 1");
    }
    1 başvuru
    private void label6_Click(object sender, EventArgs e)
    {

    }

    1 başvuru
    private void btnAdd_Click(object sender, EventArgs e)
    {
        if (txtCropName1.Text == "" || cmbField.Text == "")
        {
            MessageBox.Show("Boş alan bırakma!");
            return;
        }
        using (var conn = new NpgsqlConnection(connString))
        {
            conn.Open();

            string query = @"INSERT INTO crops (field_id, crop_name, planting_date, status)
             VALUES (
                 (SELECT field_id FROM fields WHERE field_name=@f LIMIT 1),
                 @name, @date, @status
             )";

            using (var cmd = new NpgsqlCommand(query, conn))
            {
                cmd.Parameters.AddWithValue("@f", cmbField.Text);
                cmd.Parameters.AddWithValue("@name", txtCropName1.Text);
                cmd.Parameters.AddWithValue("@date", dtPlantingDate.Value);
                cmd.Parameters.AddWithValue("@status", "New");

                cmd.ExecuteNonQuery();
            }
        }

        LoadCrops();
    }

    2 başvuru
    private void CropsForm_Load(object sender, EventArgs e)
    {
        LoadCrops();
        LoadFields();
    }

    1 başvuru
```

CropsForm.cs | FieldsForm.cs

SmartFarmingSystem | SmartFarmingSystem.CropsForm | connString

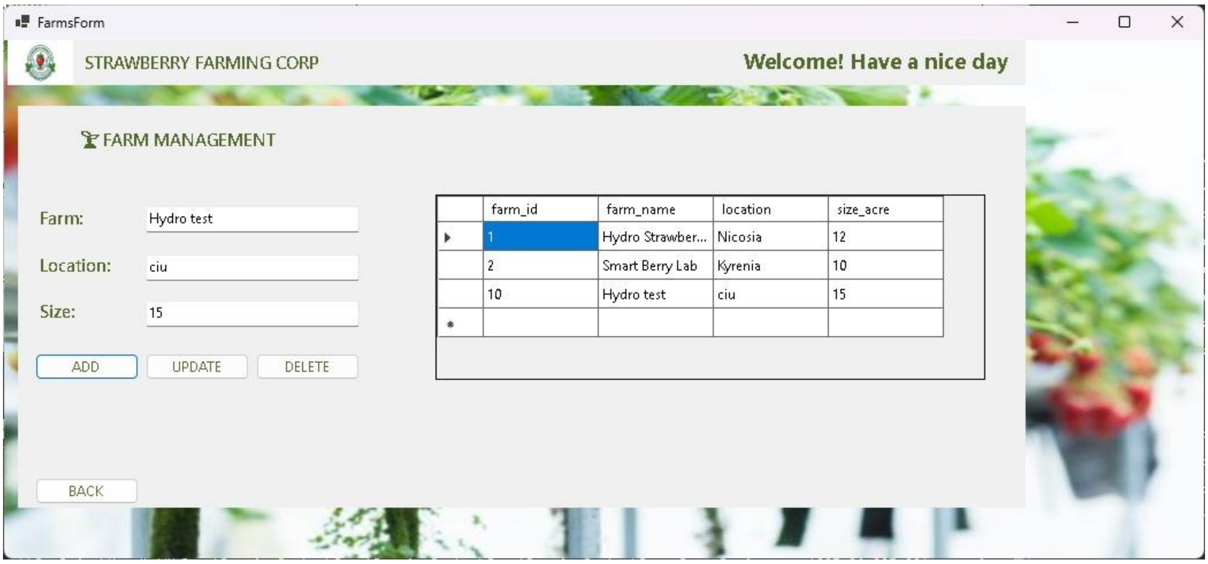
```
    1 başvuru
    private void dataGridView1_CellClick(object sender, DataGridViewCellEventArgs e)
    {
        if (e.RowIndex >= 0)
        {
            var row = dataGridView1.Rows[e.RowIndex];

            // ID
            if (row.Cells[0].Value != DBNull.Value)
                selectedCropId = Convert.ToInt32(row.Cells[0].Value);

            // Text alanları
            txtCropName1.Text = row.Cells[1].Value?.ToString();
            cmbField.Text = row.Cells[2].Value?.ToString();

            // Date fix (EN KRİTİK KISIM)
            var v = row.Cells[3].Value;

            if (v != DBNull.Value)
            {
                if (v is DateTime dt)
                    dtPlantingDate.Value = dt;

                else if (v is DateOnly d)
                    dtPlantingDate.Value = d.ToDateTime(TimeOnly.MinValue);
            }
        }
    }

    1 başvuru
    private void btnUpdate_Click(object sender, EventArgs e)
    {
        if (selectedCropId == -1)
        {
            MessageBox.Show("Önce listeden bir kayıt seç!");
            return;
        }

        using (var conn = new NpgsqlConnection(connString))
        {
            conn.Open();

            string query = @"UPDATE crops
                    SET crop_name=@name,
                        planting_date=@date,
                        field_id = (SELECT field_id FROM fields WHERE field_name=@f LIMIT 1),
                        status=@status
                    WHERE crop_id=@id";

```



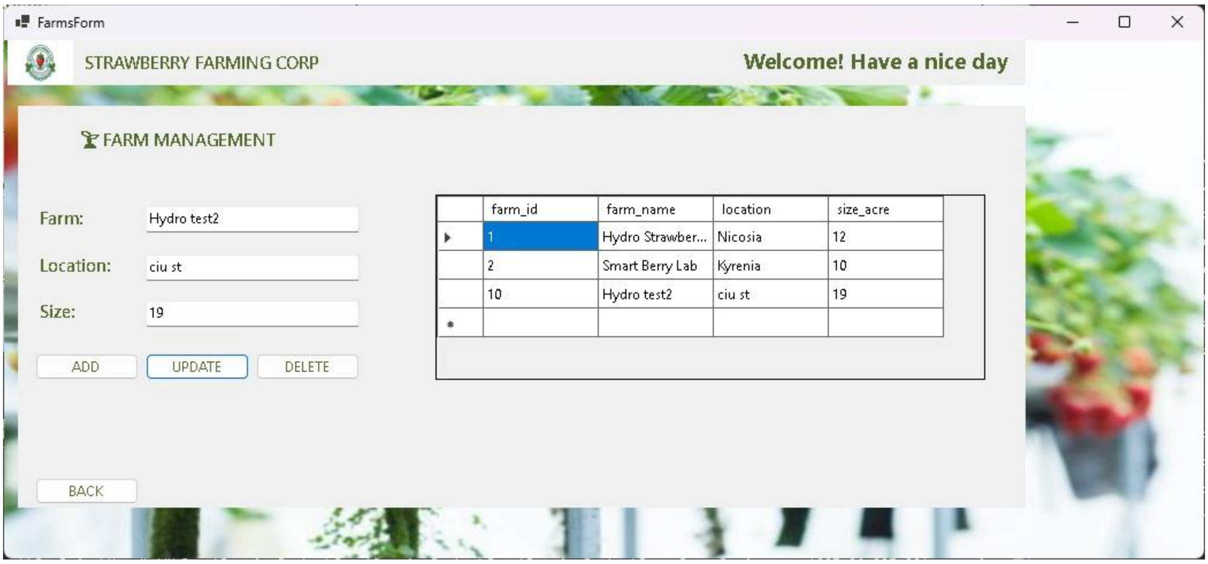
```
WHERE crop_id=@id";

using (var cmd = new NpgsqlCommand(query, conn))
{
    cmd.Parameters.AddWithValue("@name", txtCropName1.Text);
    cmd.Parameters.AddWithValue("@date", dtPlantingDate.Value);
    cmd.Parameters.AddWithValue("@f", cmbField.Text);
    cmd.Parameters.AddWithValue("@status", "Updated");
    cmd.Parameters.AddWithValue("@id", selectedCropId);

    int affected = cmd.ExecuteNonQuery();

    if (affected == 0)
        MessageBox.Show("Güncellenecek kayıt bulunamadı!");
}
}

selectedCropId = -1; // reset
LoadCrops();
}


1 başvuru
private void btnDelete_Click(object sender, EventArgs e)
{
    if (dataGridView1.CurrentRow != null)
    {
        int id = Convert.ToInt32(dataGridView1.CurrentRow.Cells[0].Value);

        using (var conn = new NpgsqlConnection(connString))
        {
            conn.Open();

            string query = "DELETE FROM crops WHERE crop_id=@id";

            using (var cmd = new NpgsqlCommand(query, conn))
            {
                cmd.Parameters.AddWithValue("@id", id);
                cmd.ExecuteNonQuery();
            }
        }

        LoadCrops();
    }
}

1 başvuru
private void dataGridView1_CellContentClick(object sender, DataGridViewCellEventArgs e)
{

}
1 başvuru
private void btnBack_Click(object sender, EventArgs e)
{
    Dashboard d = new Dashboard();
    d.Show();
    this.Close();
}
```

Farms management

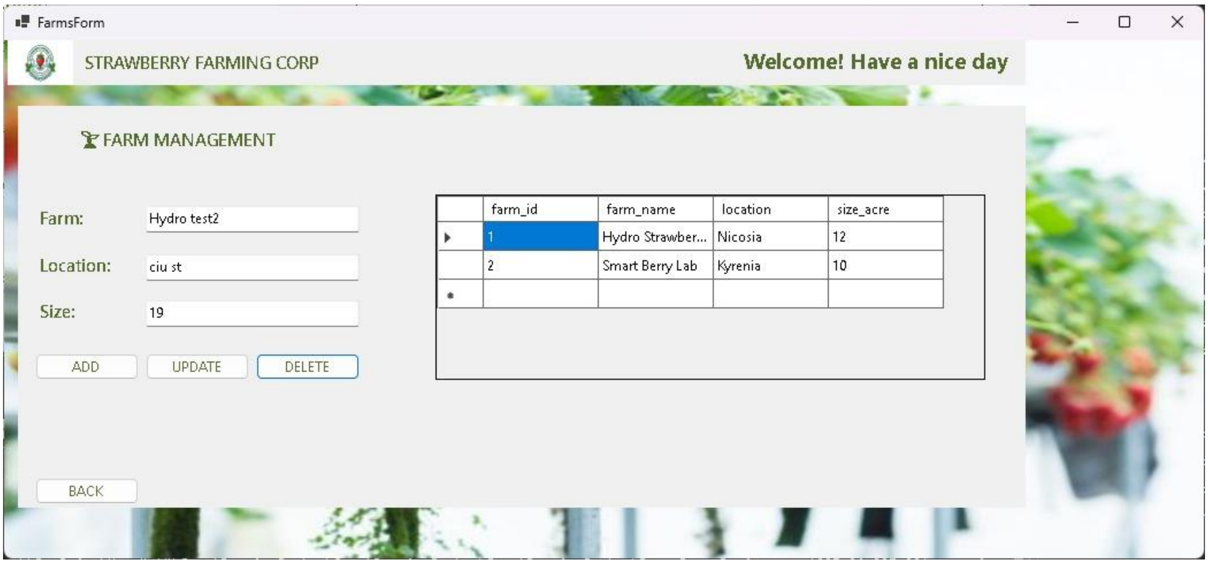
Adding farm



Updating farm



Deleting farm



FarmsForm.cs

SmartFarmingSystem | SmartFarmingSystem.FarmsForm | connString

```csharp
using Npgsql;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Text;
using System.Windows.Forms;
namespace SmartFarmingSystem
{
    5 başvuru
    public partial class FarmsForm : Form
    {
        string connString = "Host=ep-bold-surf-apre6yz3.c-7.us-east-1.aws.neon.tech;Database=neondb;Username=neondb_owner;Password=npg_nqxUsDFfP10g;SSL Mode=Require;Trust Server Certificate=true;";

        1 başvuru
        public FarmsForm()
        {
            InitializeComponent();
            this.Load += FarmsForm_Load;
            btnAdd.Click += btnAdd_Click;
            btnUpdate.Click += btnUpdate_Click;
            btnDelete.Click += btnDelete_Click;
            btnBack.Click += btnBack_Click;
            dataGridView1.CellClick += dataGridView1_CellClick;

        }

        // 🟢 LOAD
        4 başvuru
        void LoadFarms()
        {
            using (var conn = new NpgsqlConnection(connString))
            {
                conn.Open();

                string query = "SELECT * FROM farms";

                using (var da = new NpgsqlDataAdapter(query, conn))
                {
                    DataTable dt = new DataTable();
                    da.Fill(dt);
                    dataGridView1.DataSource = dt;
                }
            }
        }

        1 başvuru
        private void FarmsForm_Load(object sender, EventArgs e)
        {
            LoadFarms();
```

FarmsForm.cs

SmartFarmingSystem | SmartFarmingSystem.FarmsForm | connString

```csharp

        // ➕ ADD

        private void btnAdd_Click(object sender, EventArgs e)
        {
            if (txtFarmName.Text == "" || txtLocation.Text == "" || txtSize.Text == "")
            {
                MessageBox.Show("Boş alan bırakma!");
                return;
            }

            using (var conn = new NpgsqlConnection(connString))
            {
                conn.Open();

                string query = "INSERT INTO farms (farm_name, location, size_acre) VALUES (@n, @l, @s)";

                using (var cmd = new NpgsqlCommand(query, conn))
                {
                    cmd.Parameters.AddWithValue("@n", txtFarmName.Text);
                    cmd.Parameters.AddWithValue("@l", txtLocation.Text);
                    cmd.Parameters.AddWithValue("@s", int.Parse(txtSize.Text));

                    try
                    {
                        cmd.ExecuteNonQuery();
                    }
                    catch
                    {
                        MessageBox.Show("Bu farm zaten var!");
                    }
                }
            }

            LoadFarms();
        }

        // 🔄 UPDATE

        private void btnUpdate_Click(object sender, EventArgs e)
        {
            if (dataGridView1.CurrentRow != null)
            {
                int id = Convert.ToInt32(dataGridView1.CurrentRow.Cells[0].Value);

                using (var conn = new NpgsqlConnection(connString))
                {
                    conn.Open();

                    string query = @"UPDATE farms
                                     SET farm_name=@n, location=@l, size_acre=@s
```

FarmsForm.cs

SmartFarmingSystem | SmartFarmingSystem.FarmsForm | connString

```csharp
        // UPDATE
        1 başvuru
        private void btnUpdate_Click(object sender, EventArgs e)
        {
            if (dataGridView1.CurrentRow != null)
            {
                int id = Convert.ToInt32(dataGridView1.CurrentRow.Cells[0].Value);

                using (var conn = new NpgsqlConnection(connString))
                {
                    conn.Open();

                    string query = @"UPDATE farms
                                    SET farm_name=@n, location=@l, size_acre=@s
                                    WHERE farm_id=@id";

                    using (var cmd = new NpgsqlCommand(query, conn))
                    {
                        cmd.Parameters.AddWithValue("@n", txtFarmName.Text);
                        cmd.Parameters.AddWithValue("@l", txtLocation.Text);
                        cmd.Parameters.AddWithValue("@s", int.Parse(txtSize.Text));
                        cmd.Parameters.AddWithValue("@id", id);

                        cmd.ExecuteNonQuery();
                    }
                }

                LoadFarms();
            }
        }

        // DELETE
        1 başvuru
        private void btnDelete_Click(object sender, EventArgs e)
        {
            if (dataGridView1.CurrentRow != null)
            {
                int id = Convert.ToInt32(dataGridView1.CurrentRow.Cells[0].Value);

                using (var conn = new NpgsqlConnection(connString))
                {
                    conn.Open();

                    string query = "DELETE FROM farms WHERE farm_id=@id";

                    using (var cmd = new NpgsqlCommand(query, conn))
                    {
                        cmd.Parameters.AddWithValue("@id", id);
                        cmd.ExecuteNonQuery();
                    }
```

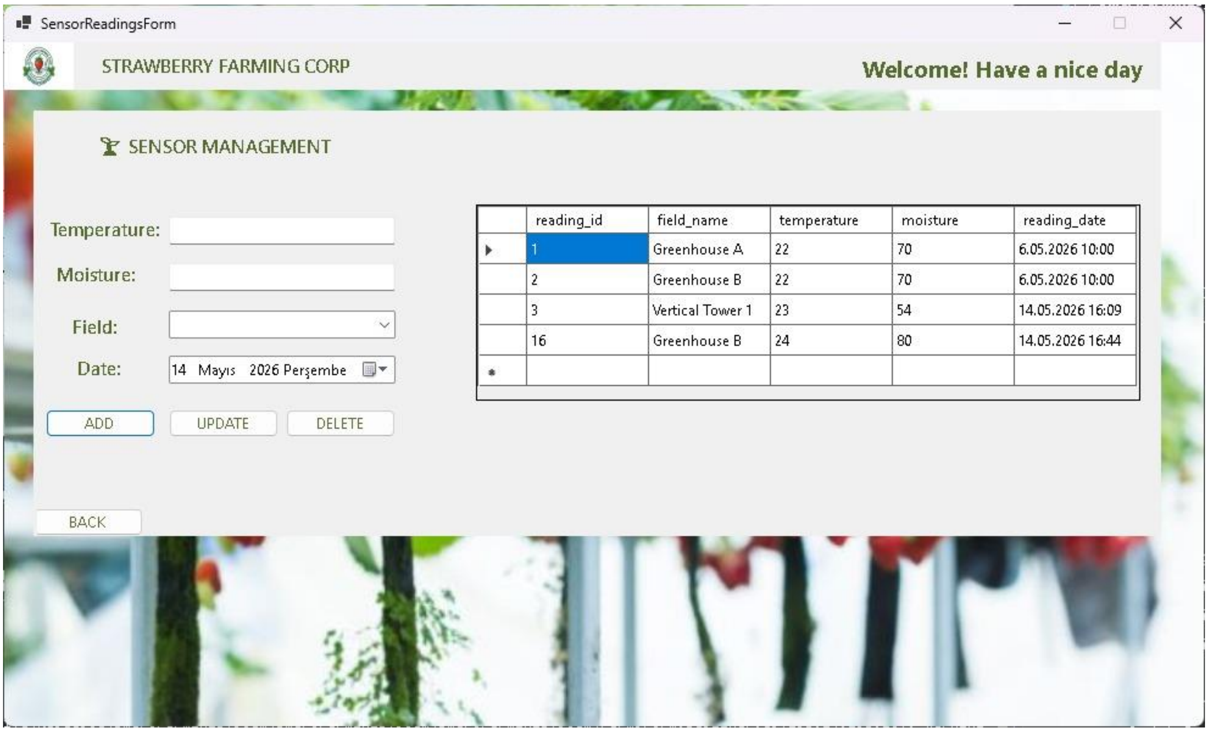
FarmsForm.cs

SmartFarmingSystem | SmartFarmingSystem.FarmsForm | connString

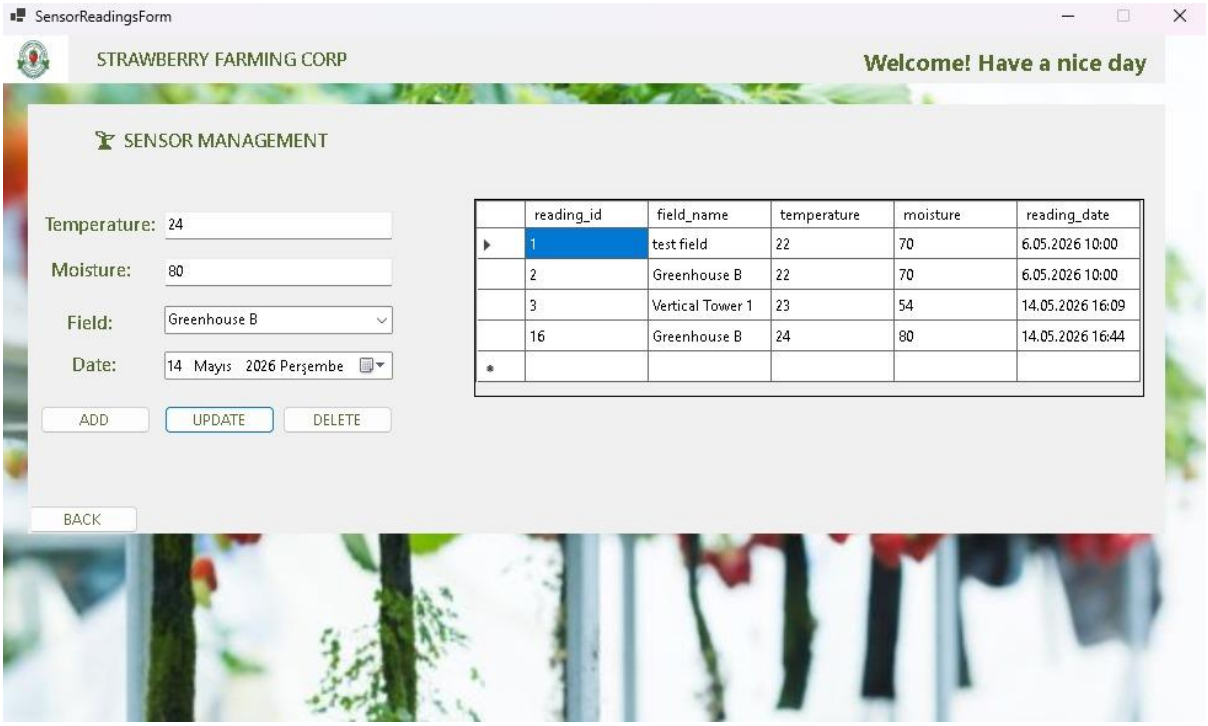
```csharp
                        cmd.ExecuteNonQuery();
                    }
                }

                LoadFarms();
            }
        }

        // GRID CLICK

        private void dataGridView1_CellClick(object sender, DataGridViewCellEventArgs e)
        {
            if (e.RowIndex >= 0)
            {
                var row = dataGridView1.Rows[e.RowIndex];

                txtFarmName.Text = row.Cells[1].Value.ToString();
                txtLocation.Text = row.Cells[2].Value.ToString();
                txtSize.Text = row.Cells[3].Value.ToString();
            }
        }

        // BACK

        private void btnBack_Click(object sender, EventArgs e)
        {
            this.Hide();
            Dashboard d = new Dashboard();
            d.Show();
            this.Close();
        }


        private void label7_Click(object sender, EventArgs e)
        {

        }

        private void label2_Click(object sender, EventArgs e)
        {

        }
    }

}
```

Sensor management

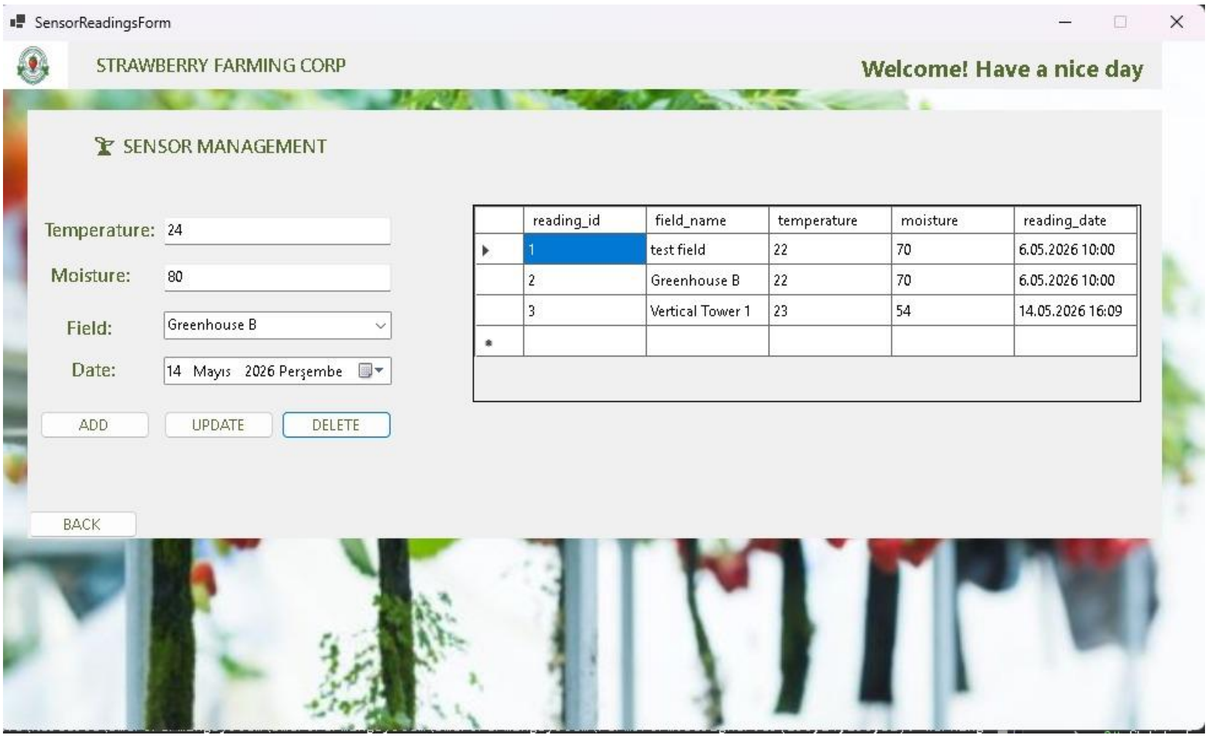
Adding sensor

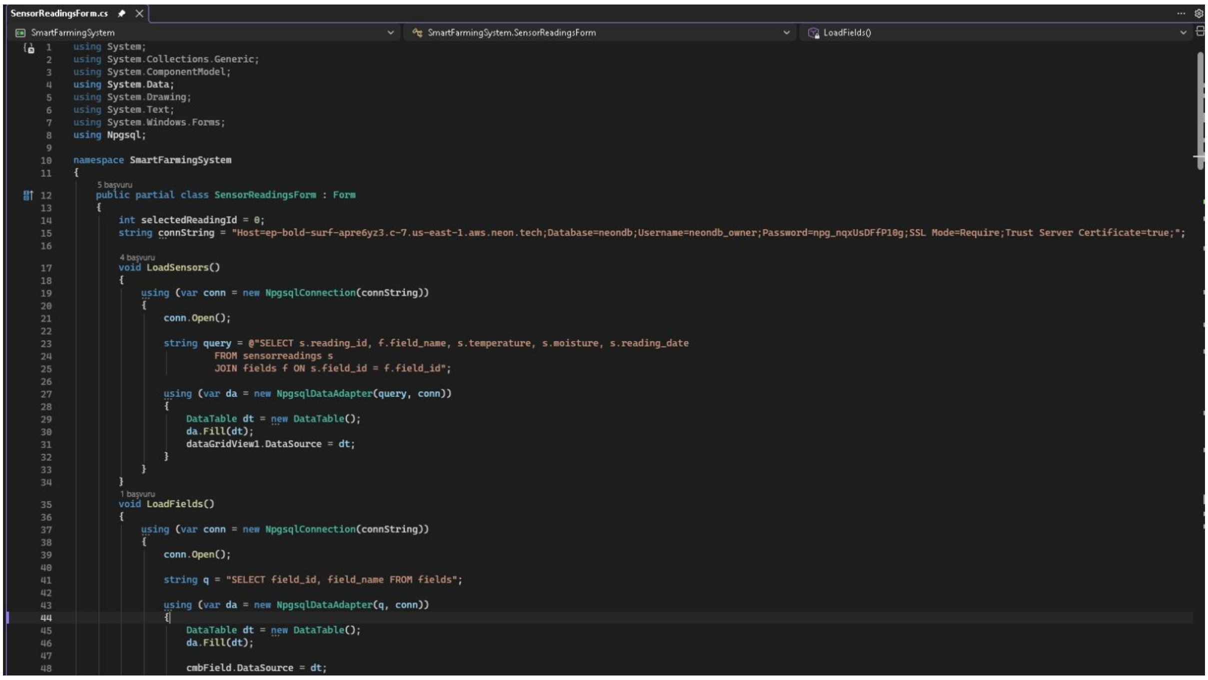


Updating sensor



Deleting sensor





```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Text;
using System.Windows.Forms;
using Npgsql;

namespace SmartFarmingSystem
{
    public partial class SensorReadingsForm : Form
    {
        int selectedReadingId = 0;
        string connString = "Host=ep-bold-surf-apre6yz3.c-7.us-east-1.aws.neon.tech;Database=neondb;Username=neondb_owner;Password=npg_nqxUsDFfP10g;SSL Mode=Require;Trust Server Certificate=true;";

        void LoadSensors()
        {
            using (var conn = new NpgsqlConnection(connString))
            {
                conn.Open();

                string query = @"SELECT s.reading_id, f.field_name, s.temperature, s.moisture, s.reading_date
                        FROM sensorreadings s
                        JOIN fields f ON s.field_id = f.field_id";

                using (var da = new NpgsqlDataAdapter(query, conn))
                {
                    DataTable dt = new DataTable();
                    da.Fill(dt);
                    dataGridView1.DataSource = dt;
                }
            }
        }
        void LoadFields()
        {
            using (var conn = new NpgsqlConnection(connString))
            {
                conn.Open();

                string q = "SELECT field_id, field_name FROM fields";

                using (var da = new NpgsqlDataAdapter(q, conn))
                {
                    DataTable dt = new DataTable();
                    da.Fill(dt);

                    cmbField.DataSource = dt;
```

SensorReadingsForm.cs

SmartFarmingSystem | SmartFarmingSystem.SensorReadingsForm | LoadFields()

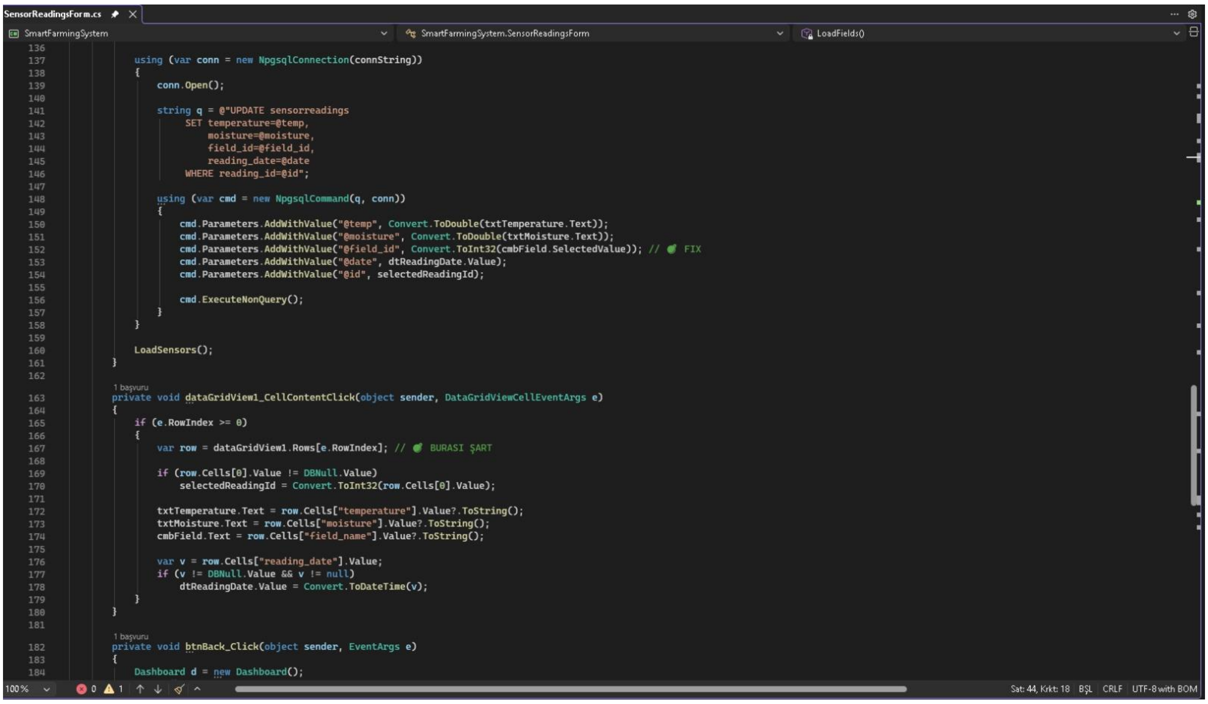
```csharp
                cmbField.DataSource = dt;
                cmbField.DisplayMember = "field_name"; // görünen
                cmbField.ValueMember = "field_id";     // arka plan (ID)
            }
        }
    }

    private void SensorReadingsForm_Load(object sender, EventArgs e)
    {
        LoadSensors();
        LoadFields();
    }

    public SensorReadingsForm()
    {
        InitializeComponent();
        this.Load += SensorReadingsForm_Load;
        this.StartPosition = FormStartPosition.CenterScreen;
        this.Size = new Size(1000, 600);
        this.FormBorderStyle = FormBorderStyle.FixedSingle;
        this.MaximizeBox = false;
    }


    private void btnAdd_Click(object sender, EventArgs e)
    {
        using (var conn = new NpgsqlConnection(connString))
        {
            conn.Open();

            string query = @"INSERT INTO sensorreadings (field_id, temperature, moisture, reading_date)
                    VALUES (
                        (SELECT field_id FROM fields WHERE field_name=@f),
                        @temp, @moist, @date
                    )";

            using (var cmd = new NpgsqlCommand(query, conn))
            {
                cmd.Parameters.AddWithValue("@f", cmbField.Text);
                cmd.Parameters.AddWithValue("@temp", int.Parse(txtTemperature.Text));
                cmd.Parameters.AddWithValue("@moist", int.Parse(txtMoisture.Text));
                cmd.Parameters.AddWithValue("@date", dtReadingDate.Value);

                cmd.ExecuteNonQuery();
            }
        }


        LoadSensors();
        txtTemperature.Clear();
```

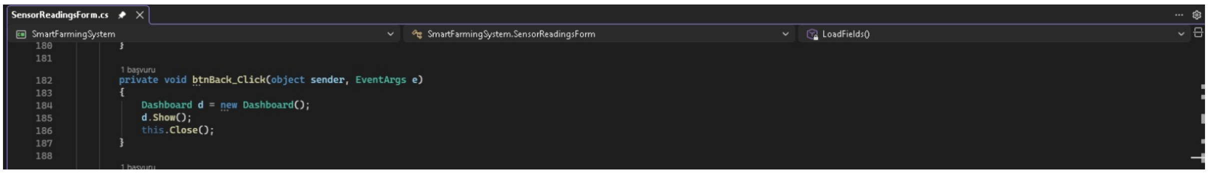
SensorReadingsForm.cs

SmartFarmingSystem | SmartFarmingSystem.SensorReadingsForm | LoadFields()

```csharp
        LoadSensors();
        txtTemperature.Clear();
        txtMoisture.Clear();
        cmbField.SelectedIndex = -1;
    }

    private void btnDelete_Click(object sender, EventArgs e)
    {

        if (dataGridView1.CurrentRow != null)
        {
            int id = Convert.ToInt32(dataGridView1.CurrentRow.Cells["reading_id"].Value);

            using (var conn = new NpgsqlConnection(connString))
            {
                conn.Open();

                string query = "DELETE FROM sensorreadings WHERE reading_id=@id";

                using (var cmd = new NpgsqlCommand(query, conn))
                {
                    cmd.Parameters.AddWithValue("@id", id);
                    cmd.ExecuteNonQuery();
                }
            }

            LoadSensors();
        }
    }

    private void btnUpdate_Click(object sender, EventArgs e)
    {
        if (selectedReadingId == 0)
        {
            MessageBox.Show("Lütfen bir kayıt seç!");
            return;
        }

        if (cmbField.SelectedValue == null)
        {
            MessageBox.Show("Field seçili değil!");
            return;
        }

        using (var conn = new NpgsqlConnection(connString))
        {
            conn.Open();

            string q = @"UPDATE sensorreadings
```

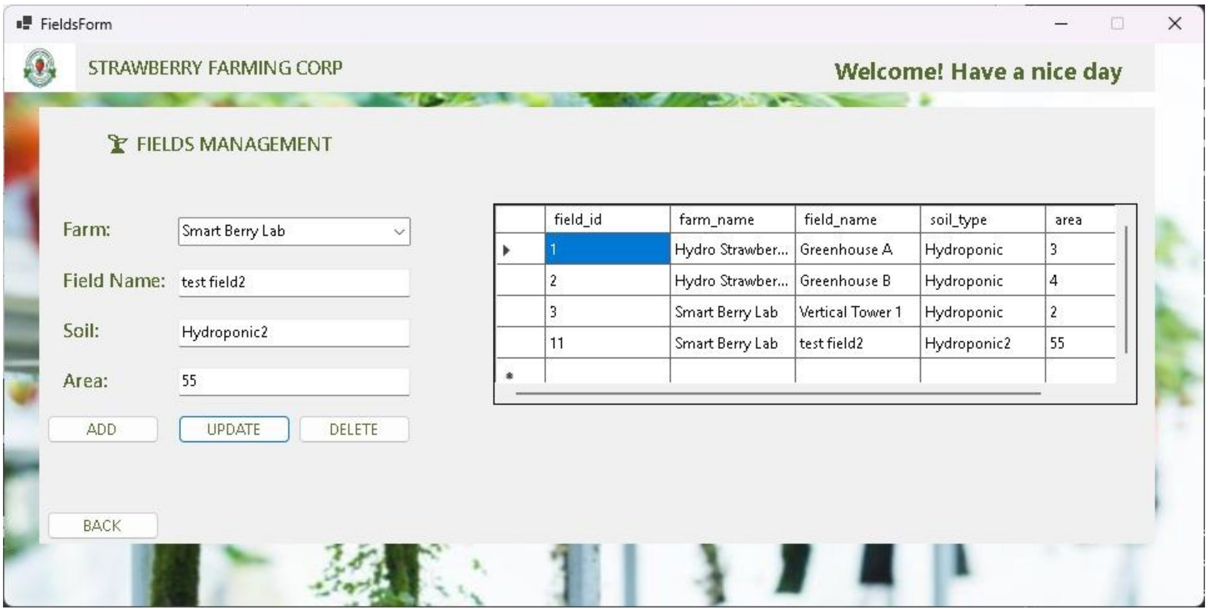
100 % | 0 | 1 | Sat: 44, Krkt: 18 | BSL | CRLF | UTF-8 with BOM



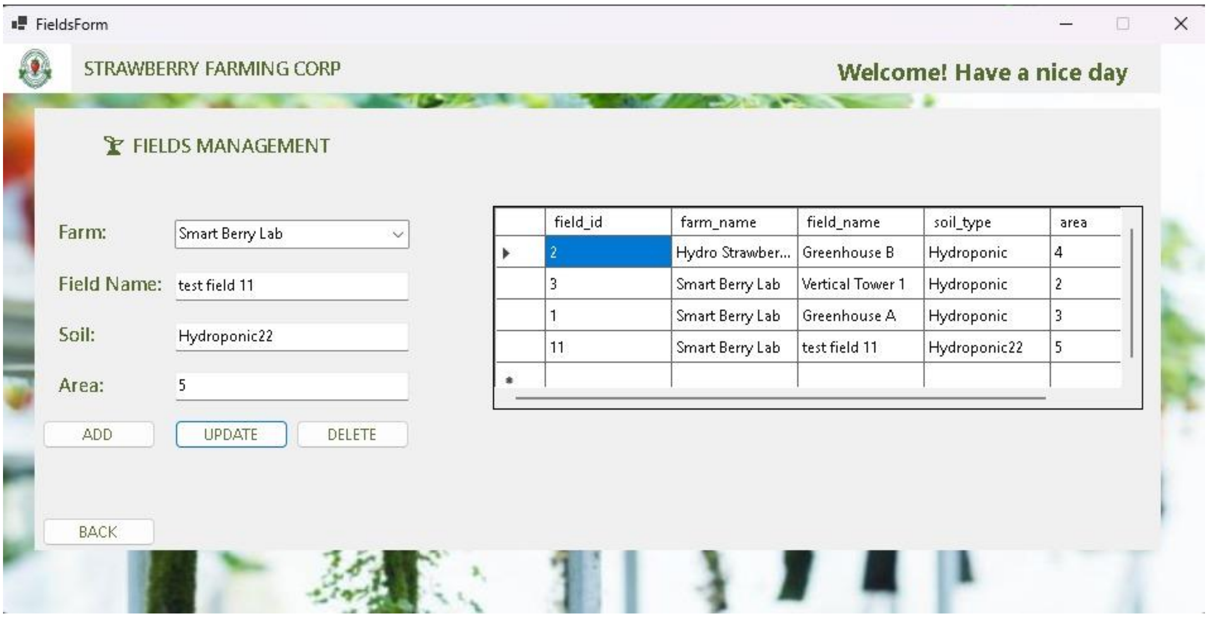


Fields Management

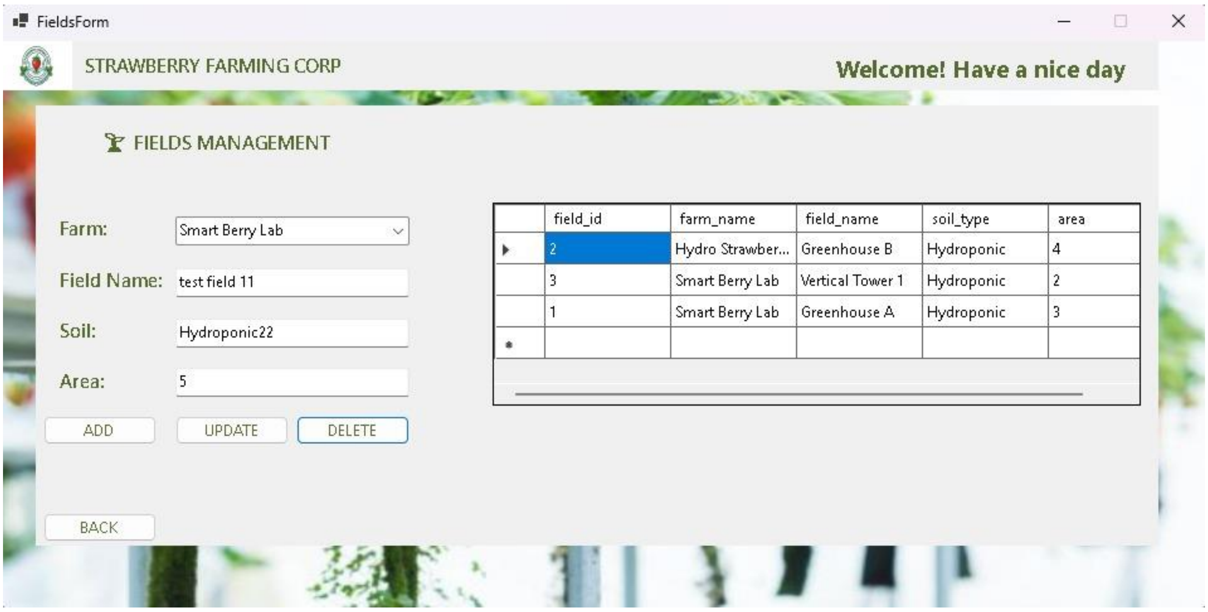
Adding Field



Updating Field



Deleting Field



FieldsForm.cs

SmartFarmingSystem | SmartFarmingSystem.FieldsForm | label2_Click(object sender, EventArgs e)

```csharp
using Npgsql;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Text;
using System.Windows.Forms;

namespace SmartFarmingSystem
{
    public partial class FieldsForm : Form
    {
        string connString = "Host=ep-bold-surf-apre6yz3.c-7.us-east-1.aws.neon.tech;Database=neondb;Username=neondb_owner;Password=npg_nqxUsDFfP10g;SSL Mode=Require;Trust Server Certificate=true;";

        public FieldsForm()
        {
            InitializeComponent();
            this.Load += FieldsForm_Load;
            this.StartPosition = FormStartPosition.CenterScreen;

            this.FormBorderStyle = FormBorderStyle.FixedSingle;
            this.MaximizeBox = false;
            btnAdd.Click += btnAdd_Click;
            btnUpdate.Click += btnUpdate_Click;
            btnDelete.Click += btnDelete_Click;
            btnBack.Click += btnBack_Click;
            dataGridView1.CellClick += dataGridView1_CellClick;
        }

        // 🟢 LOAD FARMS → dropdown

        void LoadFarms()
        {
            using (var conn = new NpgsqlConnection(connString))
            {
                conn.Open();

                string query = "SELECT farm_id, farm_name FROM farms";

                using (var da = new NpgsqlDataAdapter(query, conn))
                {
                    DataTable dt = new DataTable();
                    da.Fill(dt);

                    cmbFarm.DataSource = dt;
                    cmbFarm.DisplayMember = "farm_name";
                    cmbFarm.ValueMember = "farm_id";
                }
```

FieldsForm.cs

SmartFarmingSystem | SmartFarmingSystem.FieldsForm | label2_Click(object sender, EventArgs e)

```csharp

        // 🟢 LOAD FIELDS
        4 başvuru
        void LoadFields()
        {
            using (var conn = new NpgsqlConnection(connString))
            {
                conn.Open();

                string query = @"SELECT f.field_id, fa.farm_name, f.field_name, f.soil_type, f.area
                                 FROM fields f
                                 JOIN farms fa ON f.farm_id = fa.farm_id";

                using (var da = new NpgsqlDataAdapter(query, conn))
                {
                    DataTable dt = new DataTable();
                    da.Fill(dt);
                    dataGridView1.DataSource = dt;
                }
            }
        }

        1 başvuru
        private void FieldsForm_Load(object sender, EventArgs e)
        {
            LoadFarms();
            LoadFields();
        }

        // ➕ ADD
        1 başvuru
        private void btnAdd_Click(object sender, EventArgs e)
        {
            if (txtFieldName.Text == "")
            {
                MessageBox.Show("Boş alan bırakma!");
                return;
            }

            using (var conn = new NpgsqlConnection(connString))
            {
                conn.Open();

                string query = @"INSERT INTO fields (farm_id, field_name, soil_type, area)
                                 VALUES (@farm, @name, @soil, @area)";

                using (var cmd = new NpgsqlCommand(query, conn))
                {
                    cmd.Parameters.AddWithValue("@farm", cmbFarm.SelectedValue);
                    cmd.Parameters.AddWithValue("@name", txtFieldName.Text);
                    cmd.Parameters.AddWithValue("@soil", txtSoil.Text);
```

FieldsForm.cs

SmartFarmingSystem | SmartFarmingSystem.FieldsForm | label2_Click(object sender, EventArgs e)

```csharp
                cmd.Parameters.AddWithValue("@name", txtFieldName.Text);
                cmd.Parameters.AddWithValue("@soil", txtSoil.Text);
                cmd.Parameters.AddWithValue("@area", int.Parse(txtArea.Text));

                cmd.ExecuteNonQuery();
            }
        }

        LoadFields();
    }

    // UPDATE
    1 başvuru
    private void btnUpdate_Click(object sender, EventArgs e)
    {
        if (dataGridView1.CurrentRow != null)
        {
            int id = Convert.ToInt32(dataGridView1.CurrentRow.Cells[0].Value);

            using (var conn = new NpgsqlConnection(connString))
            {
                conn.Open();

                string query = @"UPDATE fields
                                 SET farm_id=@farm,
                                     field_name=@name,
                                     soil_type=@soil,
                                     area=@area
                                 WHERE field_id=@id";

                using (var cmd = new NpgsqlCommand(query, conn))
                {
                    cmd.Parameters.AddWithValue("@farm", cmbFarm.SelectedValue);
                    cmd.Parameters.AddWithValue("@name", txtFieldName.Text);
                    cmd.Parameters.AddWithValue("@soil", txtSoil.Text);
                    cmd.Parameters.AddWithValue("@area", int.Parse(txtArea.Text));
                    cmd.Parameters.AddWithValue("@id", id);

                    cmd.ExecuteNonQuery();
                }
            }

            LoadFields();
        }
    }

    // ❌ DELETE
    1 başvuru
    private void btnDelete_Click(object sender, EventArgs e)
    {
        if (dataGridView1.CurrentRow != null)
```

FieldsForm.cs

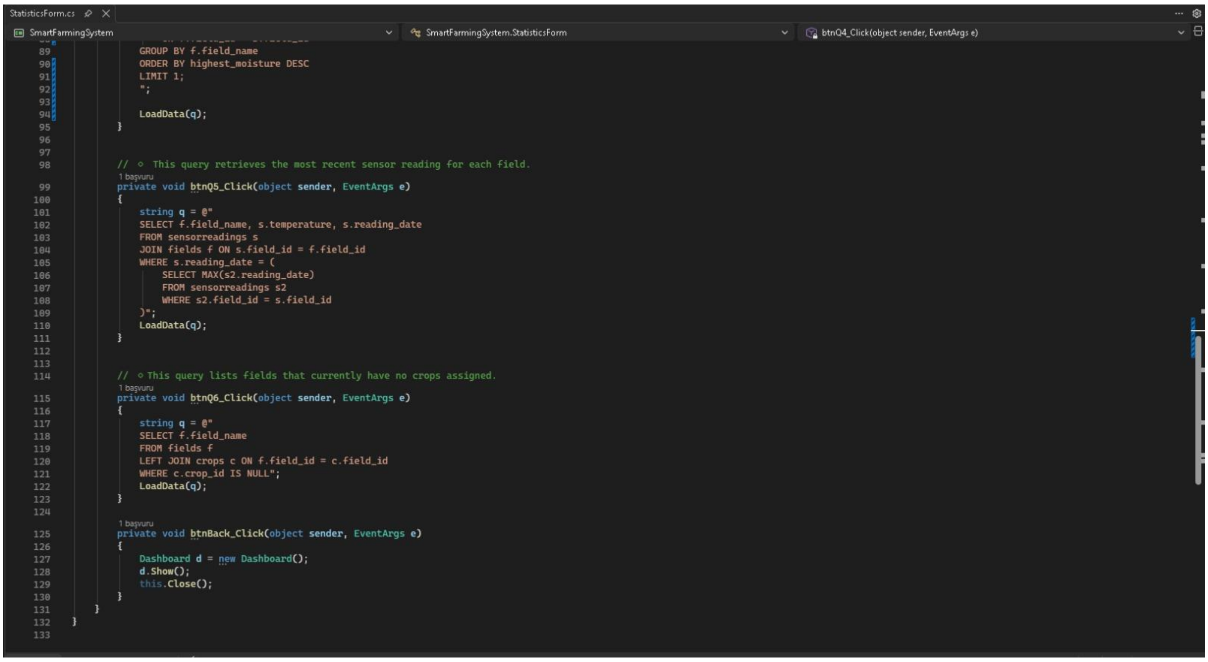
SmartFarmingSystem | SmartFarmingSystem.FieldsForm | label2_Click(object sender, EventArgs e)

```csharp

    // ❌ DELETE
    1 başvuru
    private void btnDelete_Click(object sender, EventArgs e)
    {
        if (dataGridView1.CurrentRow != null)
        {
            int id = Convert.ToInt32(dataGridView1.CurrentRow.Cells[0].Value);

            using (var conn = new NpgsqlConnection(connString))
            {
                conn.Open();

                string query = "DELETE FROM fields WHERE field_id=@id";

                using (var cmd = new NpgsqlCommand(query, conn))
                {
                    cmd.Parameters.AddWithValue("@id", id);
                    cmd.ExecuteNonQuery();
                }
            }

            LoadFields();
        }
    }

    // GRID CLICK
    1 başvuru
    private void dataGridView1_CellClick(object sender, DataGridViewCellEventArgs e)
    {
        if (e.RowIndex >= 0)
        {
            var row = dataGridView1.Rows[e.RowIndex];

            cmbFarm.Text = row.Cells[1].Value.ToString();
            txtFieldName.Text = row.Cells[2].Value.ToString();
            txtSoil.Text = row.Cells[3].Value.ToString();
            txtArea.Text = row.Cells[4].Value.ToString();
        }
    }

    // BACK
    1 başvuru
    private void btnBack_Click(object sender, EventArgs e)
    {
        this.Hide();
        Dashboard d = new Dashboard();
        d.Show();
        this.Close();
    }

```

Statistics Form

Gui screenshots of statistics are given on DML part. Statistics form codes:

```csharp
using Npgsql;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Text;
using System.Windows.Forms;

namespace SmartFarmingSystem
{
    public partial class StatisticsForm : Form
    {
        string connString = "Host=ep-bold-surf-apre6yz3.c-7.us-east-1.aws.neon.tech;Database=neondb;Username=neondb_owner;Password=npg_nqxUsDFfP10g;SSL Mode=Require;Trust Server Certificate=true;";
        int selectedCropId = -1;
        public StatisticsForm()
        {
            InitializeComponent();
        }

        void LoadData(string query)
        {
            using (var conn = new NpgsqlConnection(connString))
            {
                conn.Open();

                using (var da = new NpgsqlDataAdapter(query, conn))
                {
                    DataTable dt = new DataTable();
                    da.Fill(dt);
                    dataGridView1.DataSource = dt;
                }
            }
        }

        //This query shows fields with an average temperature higher than the overall system average.
        private void btnQ1_Click(object sender, EventArgs e)
        {
            string q = @"
            SELECT f.field_name, AVG(s.temperature) AS avg_temp
            FROM sensorreadings s
            JOIN fields f ON s.field_id = f.field_id
            GROUP BY f.field_name
            HAVING AVG(s.temperature) > (
                SELECT AVG(temperature) FROM sensorreadings
            )";

            LoadData(q);
        }


        // ◇ This query calculates the total number of crops in each farm using multiple table joins.
        private void btnQ2_Click(object sender, EventArgs e)
        {
            string q = @"
            SELECT fa.farm_name, COUNT(c.crop_id) AS total_crops
            FROM farms fa
            JOIN fields f ON fa.farm_id = f.farm_id
            JOIN crops c ON f.field_id = c.field_id
            GROUP BY fa.farm_name
            ORDER BY total_crops DESC";
            LoadData(q);
        }


        // ◇  This query displays the minimum and maximum temperature recorded in each field.
        private void btnQ3_Click(object sender, EventArgs e)
        {
                  string q = @"
            SELECT f.field_name,
                   MAX(s.temperature) AS max_temp,
                   MIN(s.temperature) AS min_temp
            FROM sensorreadings s
            JOIN fields f ON s.field_id = f.field_id
            GROUP BY f.field_name";
            LoadData(q);
        }


        // ◇ This query identifies fields that frequently experience low moisture levels.
        private void btnQ4_Click(object sender, EventArgs e)
        {
            string q = @"
            SELECT f.field_name,
                   MAX(s.moisture) AS highest_moisture
            FROM fields f
            JOIN Sensorreadings s
                ON f.field_id = s.field_id
            GROUP BY f.field_name
            ORDER BY highest_moisture DESC
            LIMIT 1;
```




```
            GROUP BY f.field_name
            ORDER BY highest_moisture DESC
            LIMIT 1;
            ";

            LoadData(q);
        }


        // ◇ This query retrieves the most recent sensor reading for each field.
        1 başvuru
        private void btnQ5_Click(object sender, EventArgs e)
        {
            string q = @"
            SELECT f.field_name, s.temperature, s.reading_date
            FROM sensorreadings s
            JOIN fields f ON s.field_id = f.field_id
            WHERE s.reading_date = (
                SELECT MAX(s2.reading_date)
                FROM sensorreadings s2
                WHERE s2.field_id = s.field_id
            )";
            LoadData(q);
        }


        // ◇ This query lists fields that currently have no crops assigned.
        1 başvuru
        private void btnQ6_Click(object sender, EventArgs e)
        {
            string q = @"
            SELECT f.field_name
            FROM fields f
            LEFT JOIN crops c ON f.field_id = c.field_id
            WHERE c.crop_id IS NULL";
            LoadData(q);
        }

        1 başvuru
        private void btnBack_Click(object sender, EventArgs e)
        {
            Dashboard d = new Dashboard();
            d.Show();
            this.Close();
        }
    }
}
```

6) GitHub Link:

[Strawberry farming corp.]